GNNDrive

Reducing Memory Contention and I/O Congestion for Disk-based GNN Training

Qisheng Jiang · Lei Jia · Chundong Wang
ShanghaiTech University, Shanghai, China

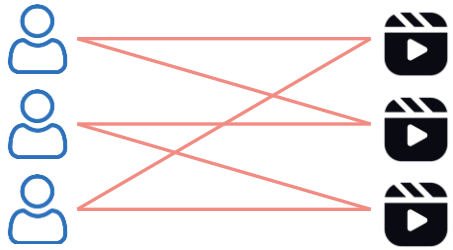
ICPP '24 · The 53rd International Conference on Parallel Processing · Gotland, Sweden · August 12–15, 2024

Presenter: Seungjong Lee · Chung-Ang University

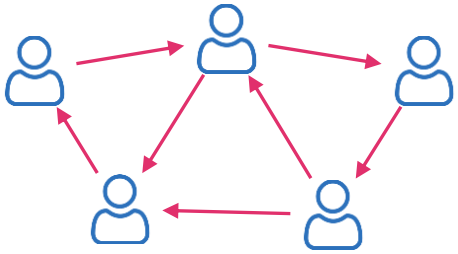
Where GNNs Are Used : Graph Data Is Everywhere

Users, videos, and products naturally form **graphs**, and one trained GNN serves many tasks.

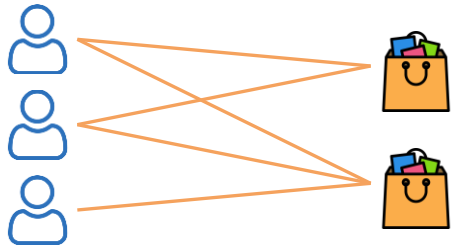
watch history



who follows whom



purchase history



Training

learns node embeddings

Recommendation

Next video to watch, accounts to follow, products to buy

Classification

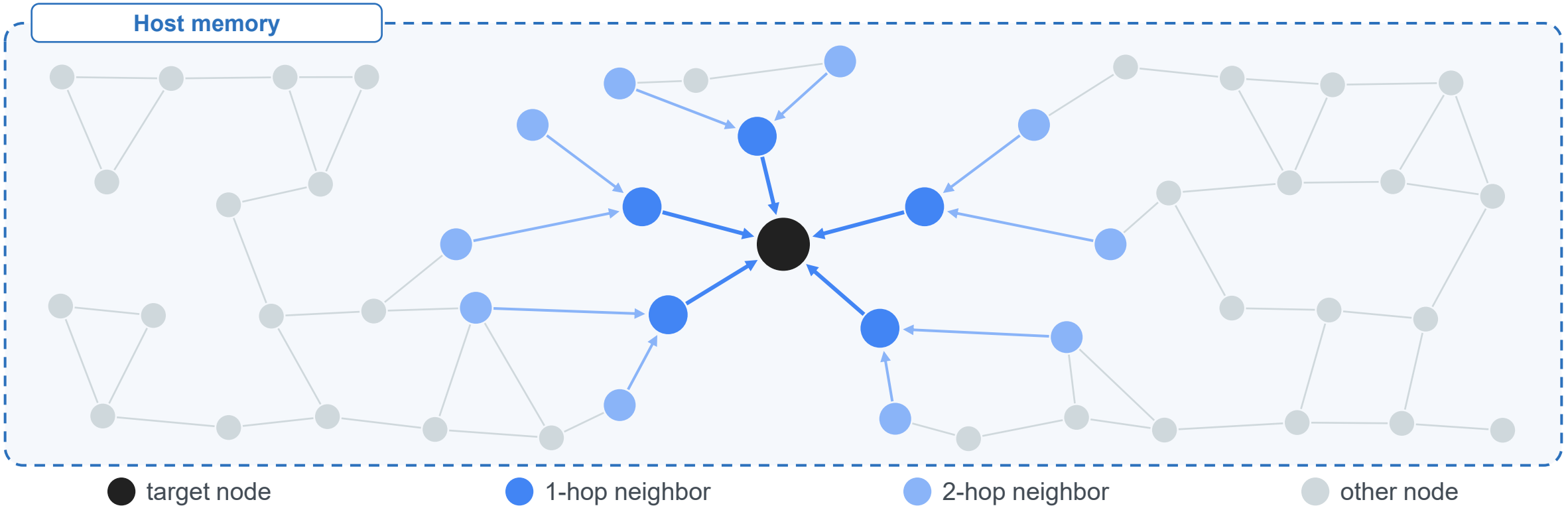
Label each node
e.g., fake or spam accounts, product categories

Grouping

Cluster similar nodes
e.g., friend communities, customer segments

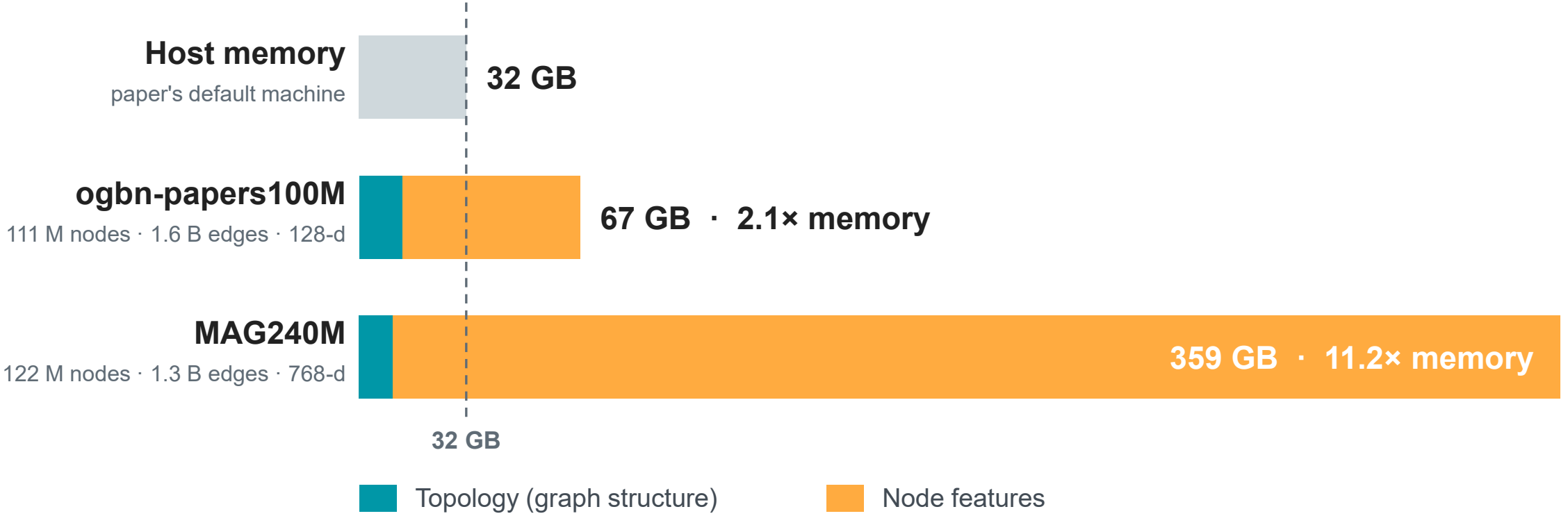
How GNN Training Works

- A GNN learns each node's embedding by **aggregating information from its neighbors**, hop by hop.
- Sampled neighbors can be any node, so **every node's features must be in memory**.



But Real Graphs Have Outgrown Memory

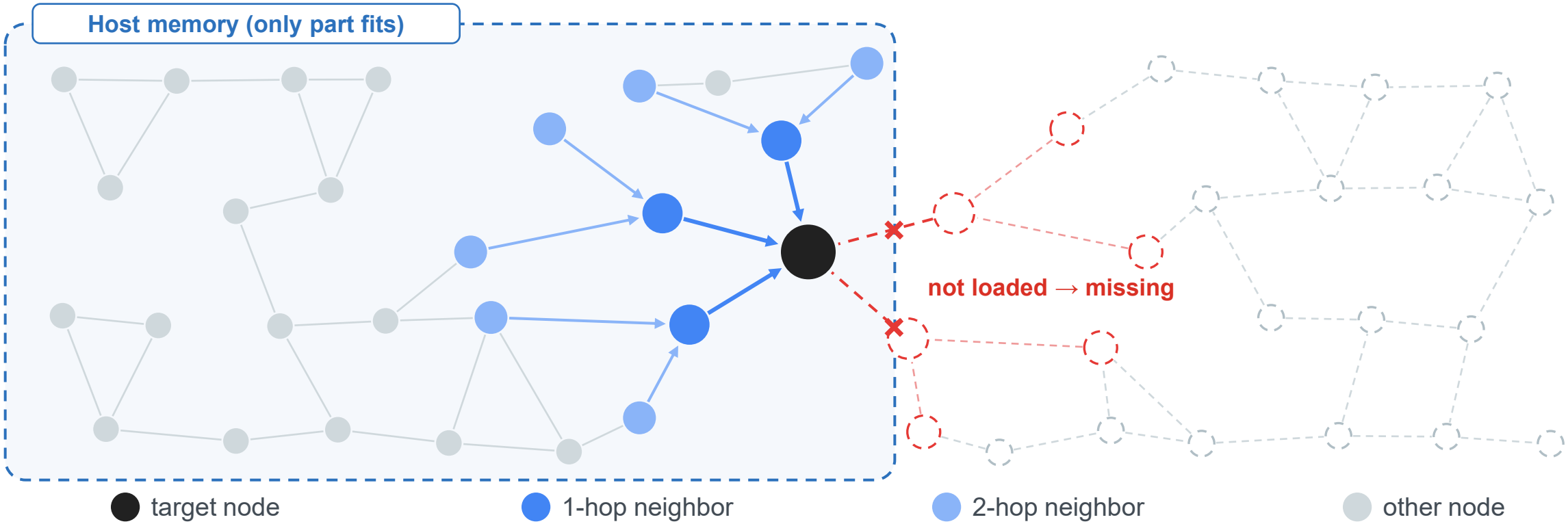
Datasets nowadays are far larger than the memory of an ordinary machine.



Node features dominate the volume and the data no longer fits in memory.

GNN Training Wants the Whole Graph in Memory

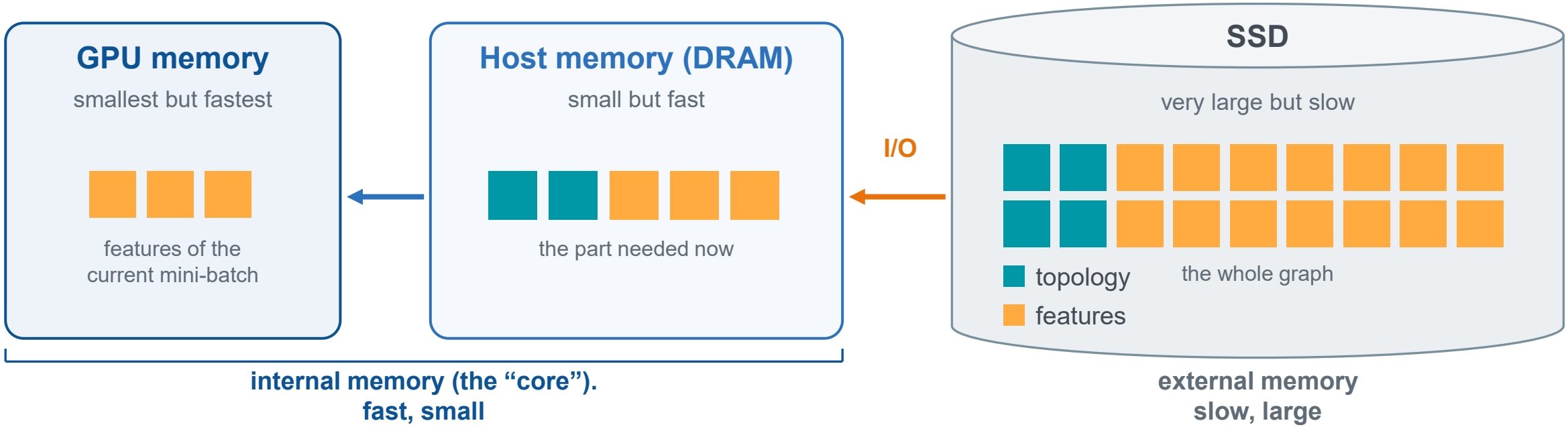
- If only part of the graph fits in memory, **edges to unloaded nodes are cut**.
- The neighbors a node needs go missing, so aggregation produces **incomplete embeddings**.



So training systems traditionally keep the whole graph's topology and features in memory.

Out-of-Core Training: Bring Data In from Disk on Demand

Keep the graph on SSD and load only what each step needs, so I/O becomes part of training.

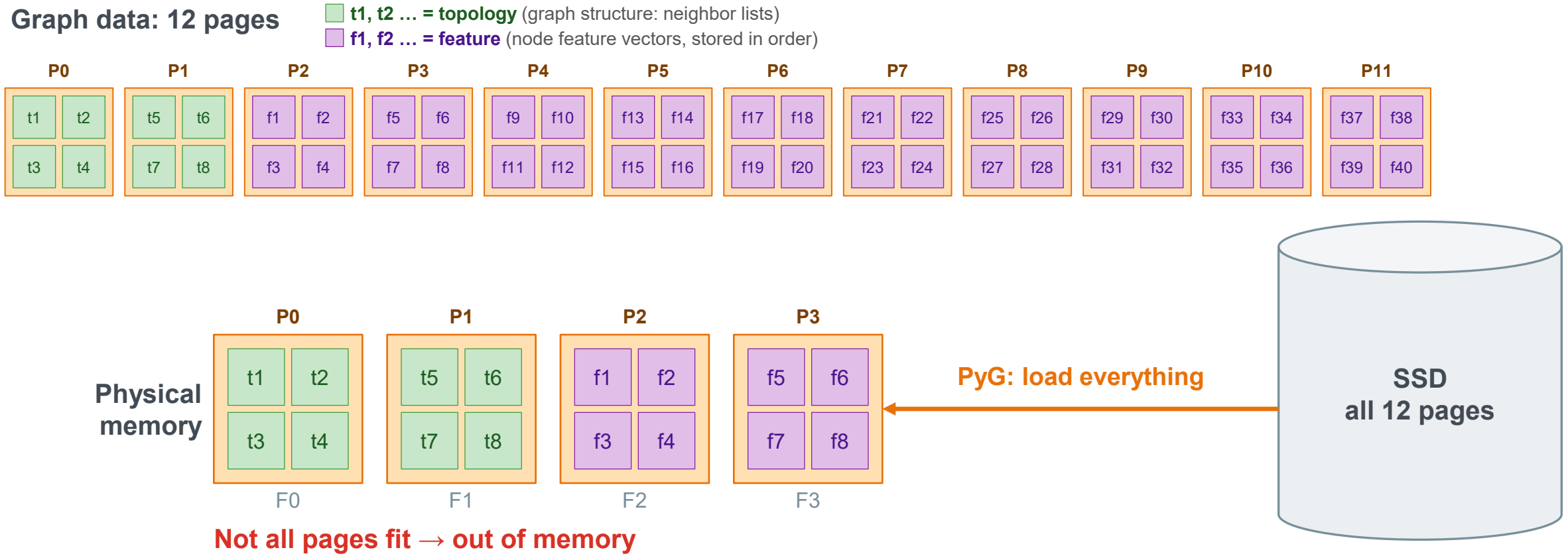


Out-of-core: **data that exceeds main memory lives on disk and is explicitly moved in as needed.**

J. S. Vitter, “External Memory Algorithms and Data Structures,” ACM Computing Surveys 33(2), 2001 · S. Toledo, DIMACS Series, 1999

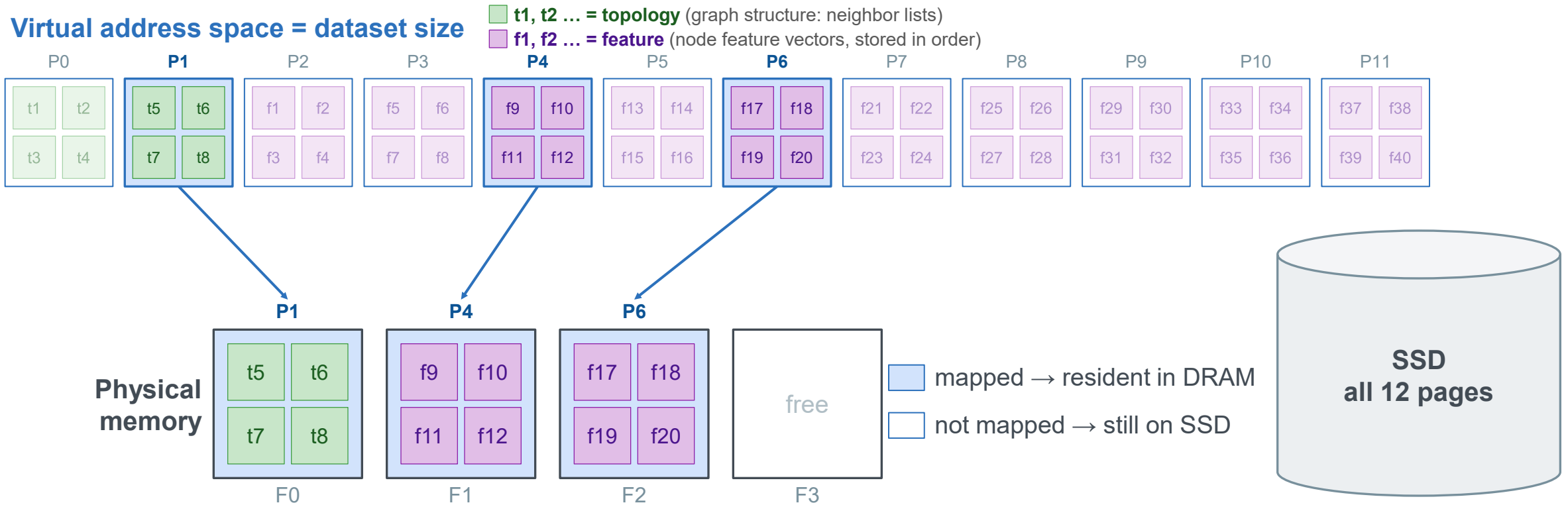
① PyG+ : Out-of-Core Training via Memory Mapping

Plain PyG assumes the whole dataset is in memory, so a larger graph simply does not fit.



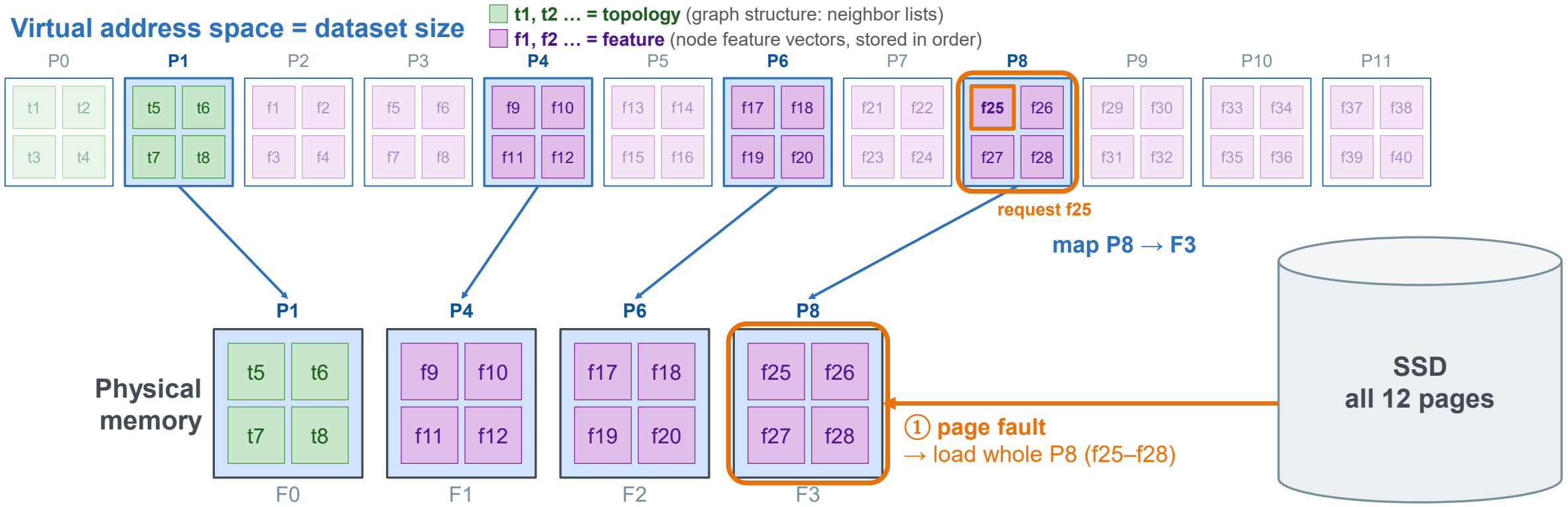
① PyG+ : Out-of-Core Training via Memory Mapping

PyG+ memory-maps the data: a virtual address space as large as the dataset.



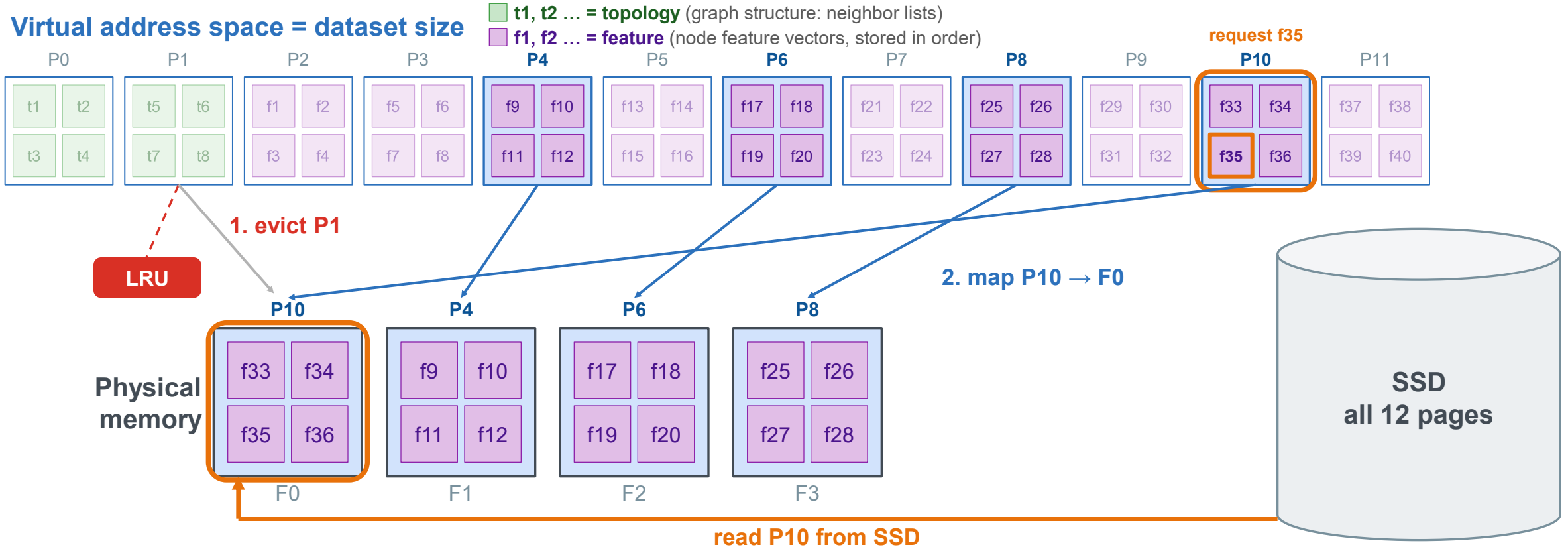
① PyG+ : Out-of-Core Training via Memory Mapping

Touching an unmapped page triggers a page fault: the OS reads it from SSD and maps it.



① PyG+ : Out-of-Core Training via Memory Mapping

When memory is full, the OS evicts a page by its own policy (LRU-like) to make room.



PyG+ runs unmodified training code, but the OS, not the GNN system, decides what stays in memory.

② Ginex : A GNN-Aware Cache Plan Instead of the OS Cache

To reduce I/O, Ginex groups mini-batches into a superbatch, samples them all first, then plans the cache.



Superbatch
1,500 mini-batches (default)

1 Sample the whole superbatch first

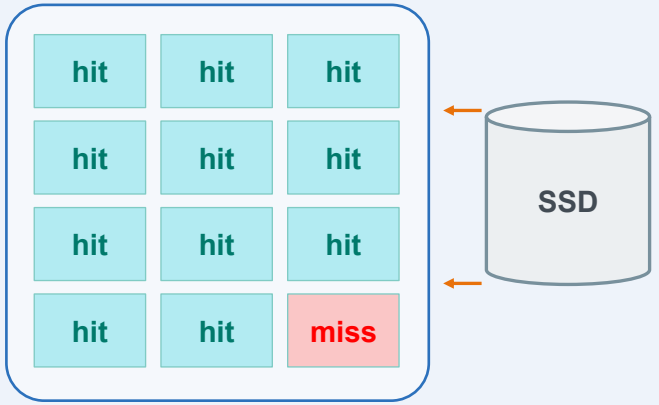
B1	3 7 12 25
B2	7 9 21 40
B3	3 21 33 52

2 Plan the cache with future knowledge

	B1	B2	B3
feature 3	keep	evict	keep
feature 7	keep	keep	evict
feature 21	load	keep	keep

Belady-optimal:
evict what is needed furthest in the future

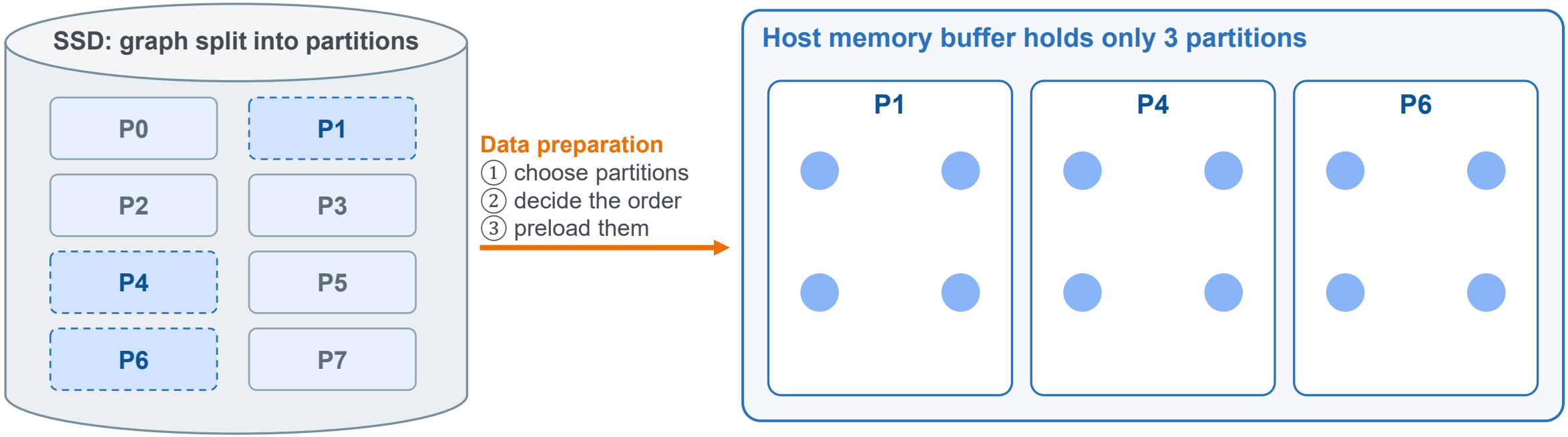
3 Extract & train following the plan

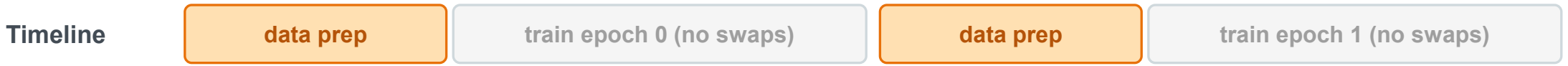


Known future accesses → optimal replacement → far fewer I/Os during training.

③ MariusGNN : Train Only on Partitions Already in Memory

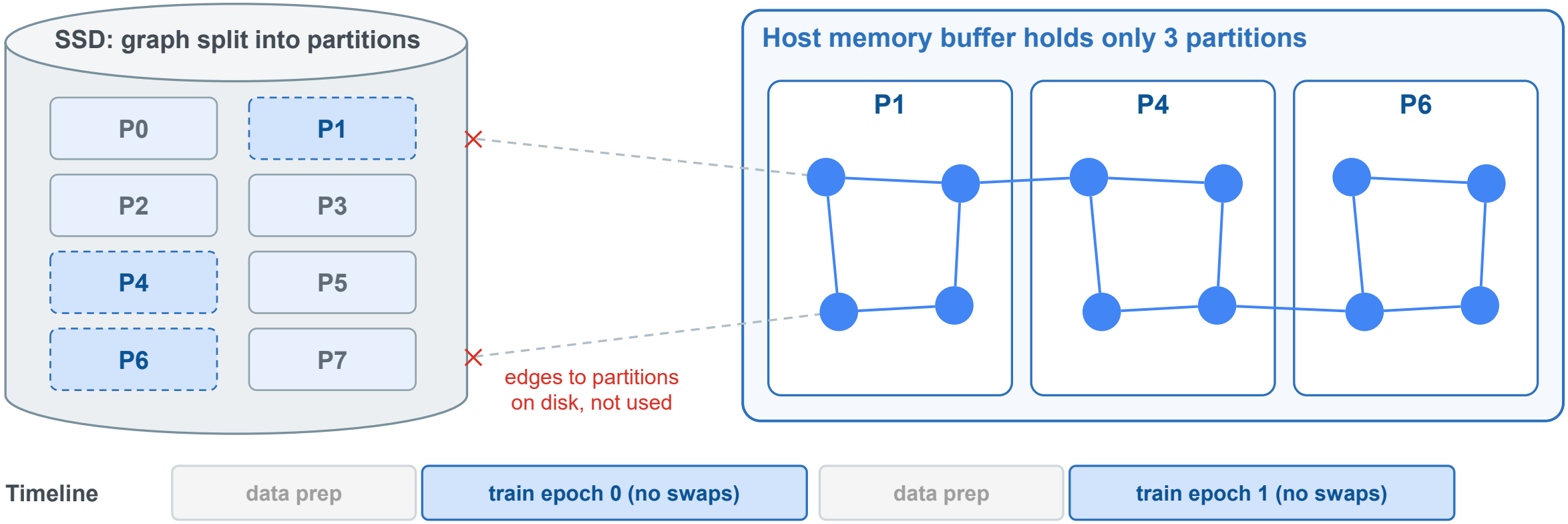
I/O is the main cost → before each epoch, pick a few partitions, order them, and preload them.





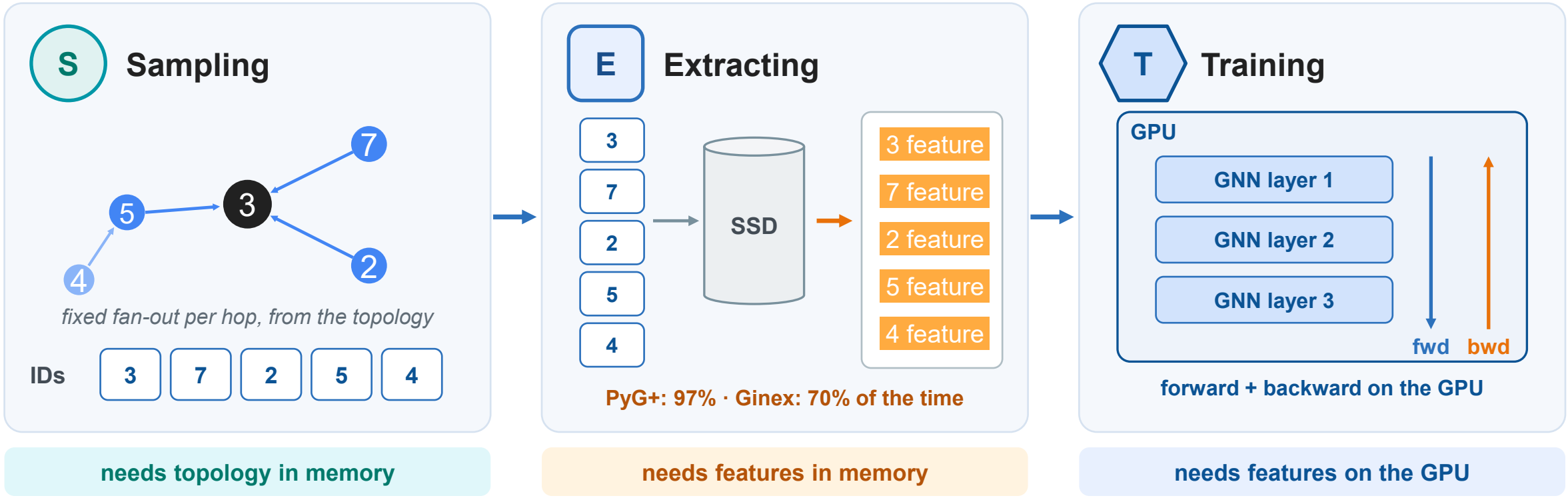
③ MariusGNN : Train Only on Partitions Already in Memory

During the epoch it samples and trains only inside the loaded partitions, with almost no I/O.



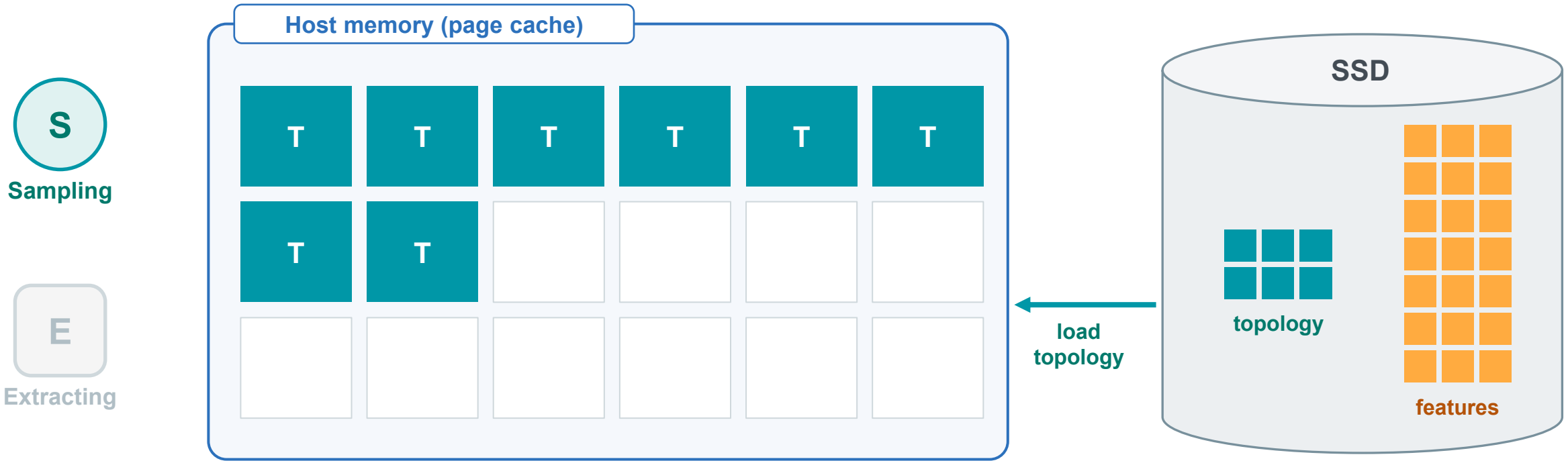
Background of Two Limitations : SET Structure

Every mini-batch runs "Sample → Extract → Train" : S needs topology, E needs features, T needs the GPU.



Limitation ① Memory Contention Problem Between S and E

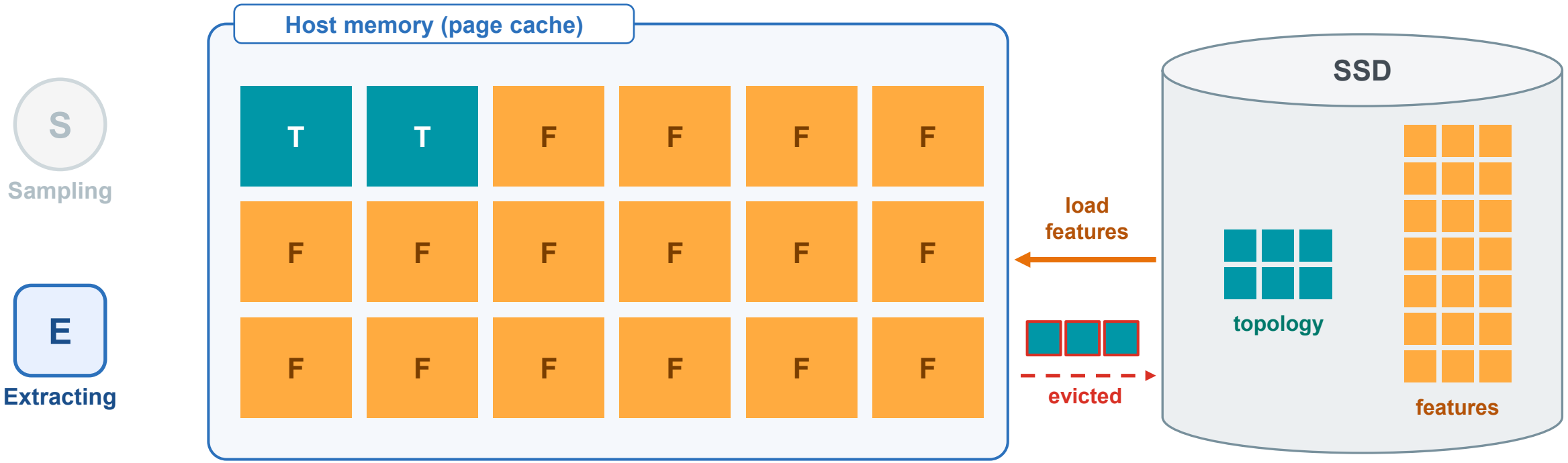
Sampling fills host memory with the topology it needs.



S brings in the topology pages it needs.

Limitation ① Memory Contention Problem Between S and E

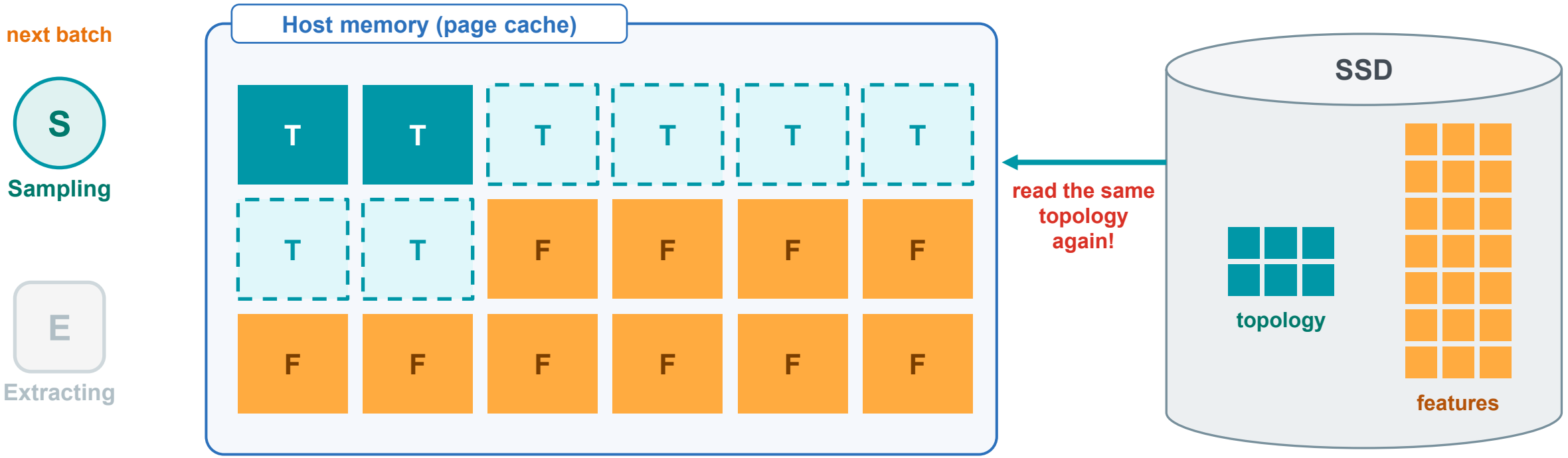
Extracting then floods memory with features and pushes the topology out.



Features push the topology out.

Limitation ① Memory Contention Problem Between S and E

The next mini-batch's sampling must reload the same topology again and again.

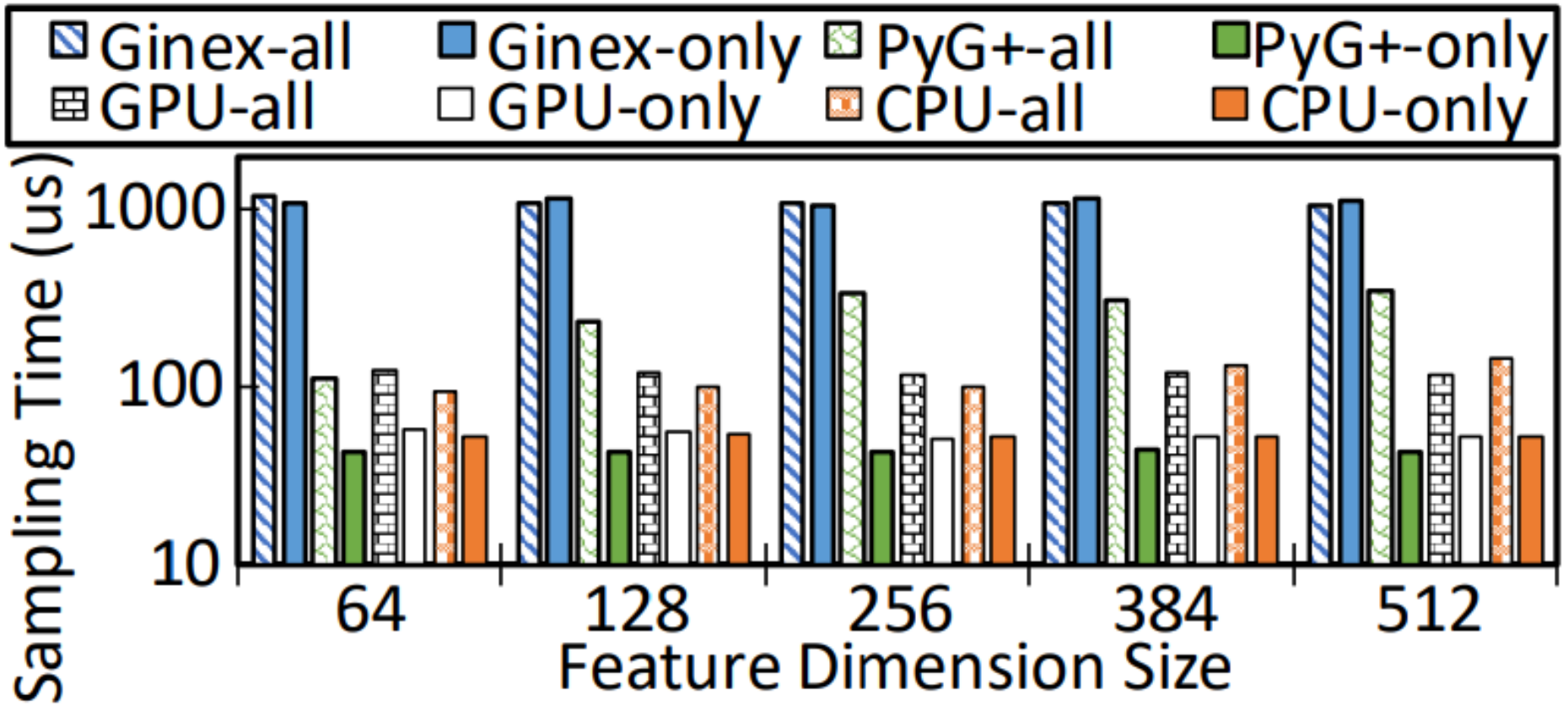


The same topology must be read from SSD again.

S and E fight over the same memory → sampling keeps re-reading the topology.

Limitation ① Memory Contention Problem Between S and E

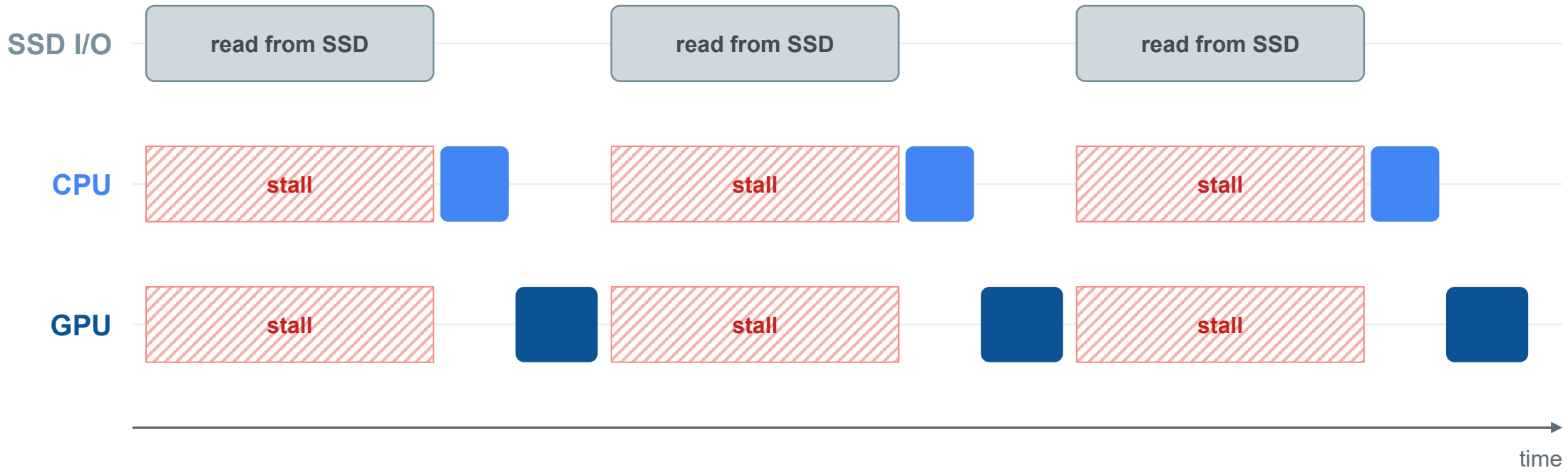
Measured: PyG+ samples 5.4× slower when the whole pipeline runs (-all) than when it only samples (-only).



Limitation ② I/O Congestion Stalls the CPU and GPU

While data is read synchronously from SSD, the CPU and GPU sit idle.

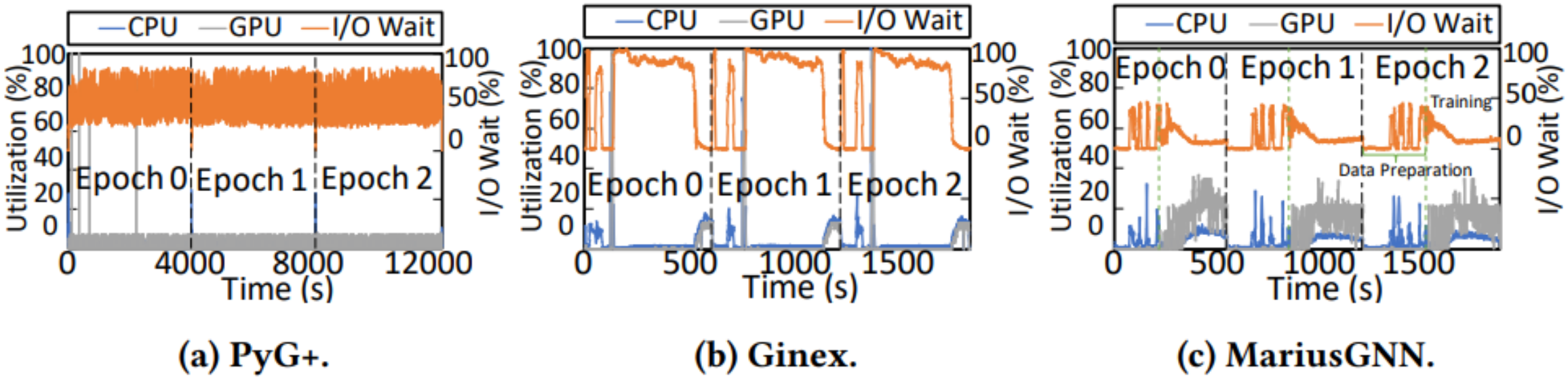
- **Synchronous read:** the thread blocks until data arrives
- **GPU waits** for the features of its next mini-batch



Long synchronous I/O waits leave the CPU and GPU idle most of the time.

Limitation ② I/O Congestion Stalls the CPU and GPU

While data is read synchronously from SSD, the CPU and GPU sit idle.

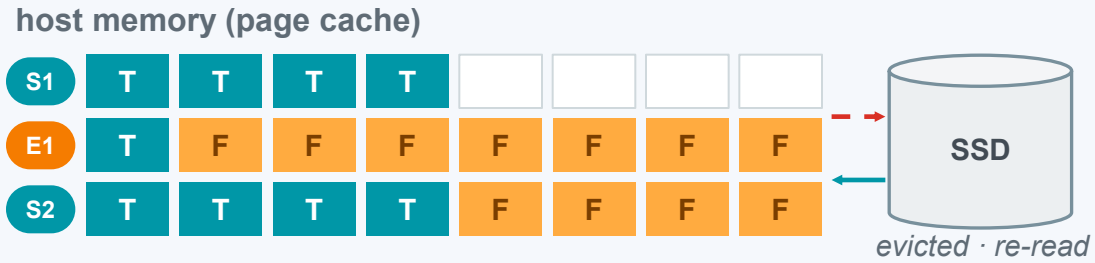


(a) PyG+. (b) Ginex. (c) MariusGNN.

GNNDrive : Two Answers to Two Limitations

1 Memory contention

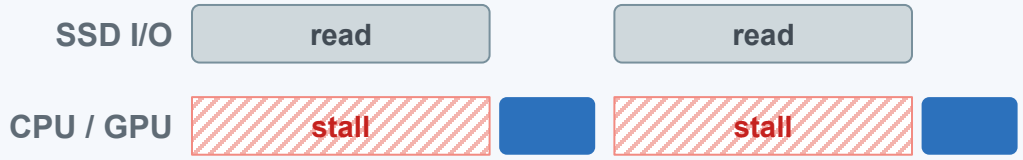
S and E fight over the same host memory



- Features push out the topology that sampling needs
- **PyG+: 5.4× slower sampling** (-all vs -only)

2 I/O congestion

Synchronous SSD reads stall the CPU and GPU



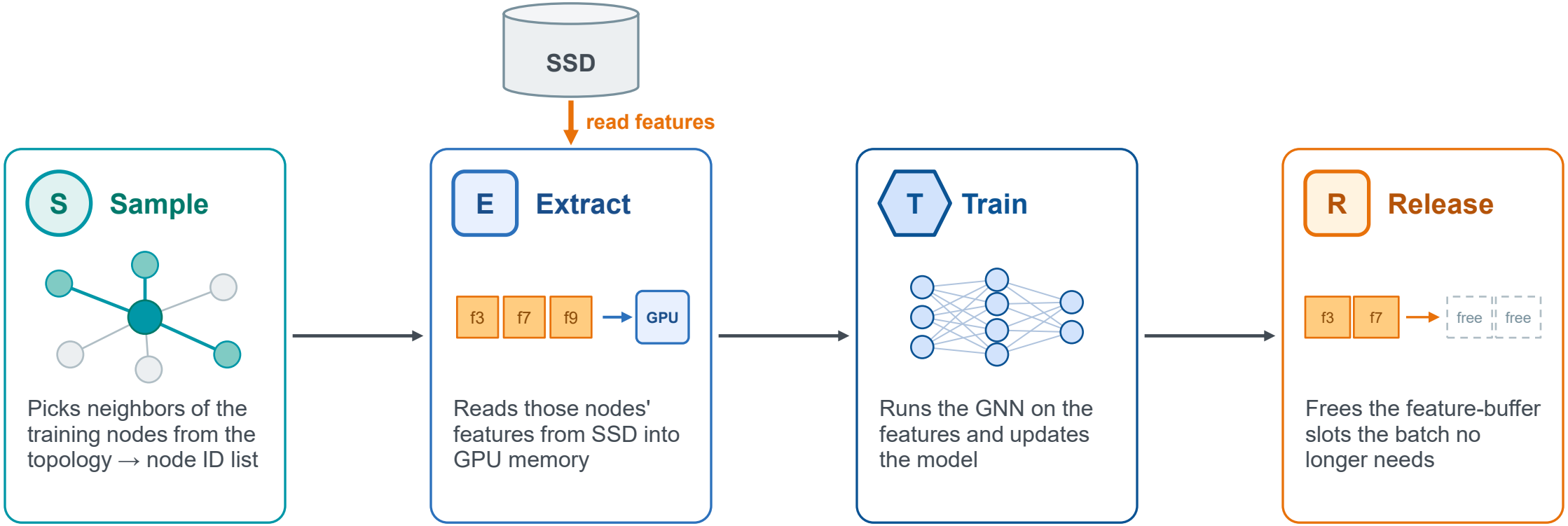
- **PyG+:** I/O wait 25–85% the whole run
- **Ginex / MariusGNN:** superbatch prep / data prep (≤ 46%)

① **Memory contention** → **solved by limiting the memory that features can use**

② **I/O congestion** → **solved by designing the stages to run asynchronously**

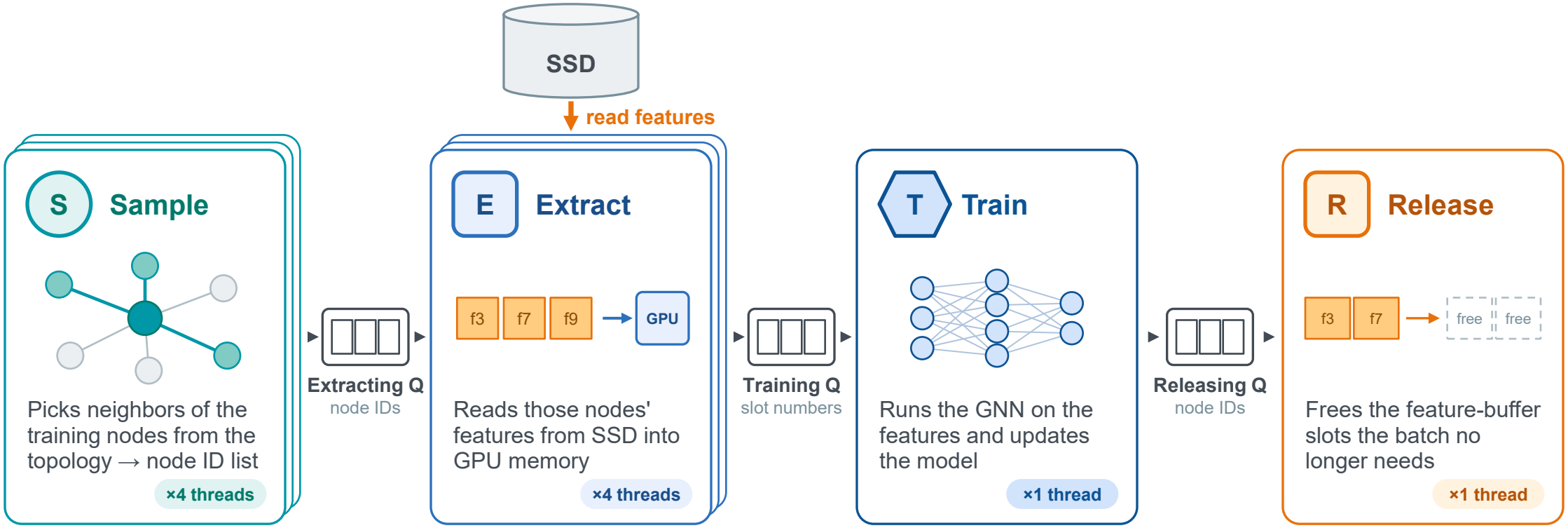
Split Training into Four Independent Stages

Each stage does one job on its own: sample, extract, train, release.



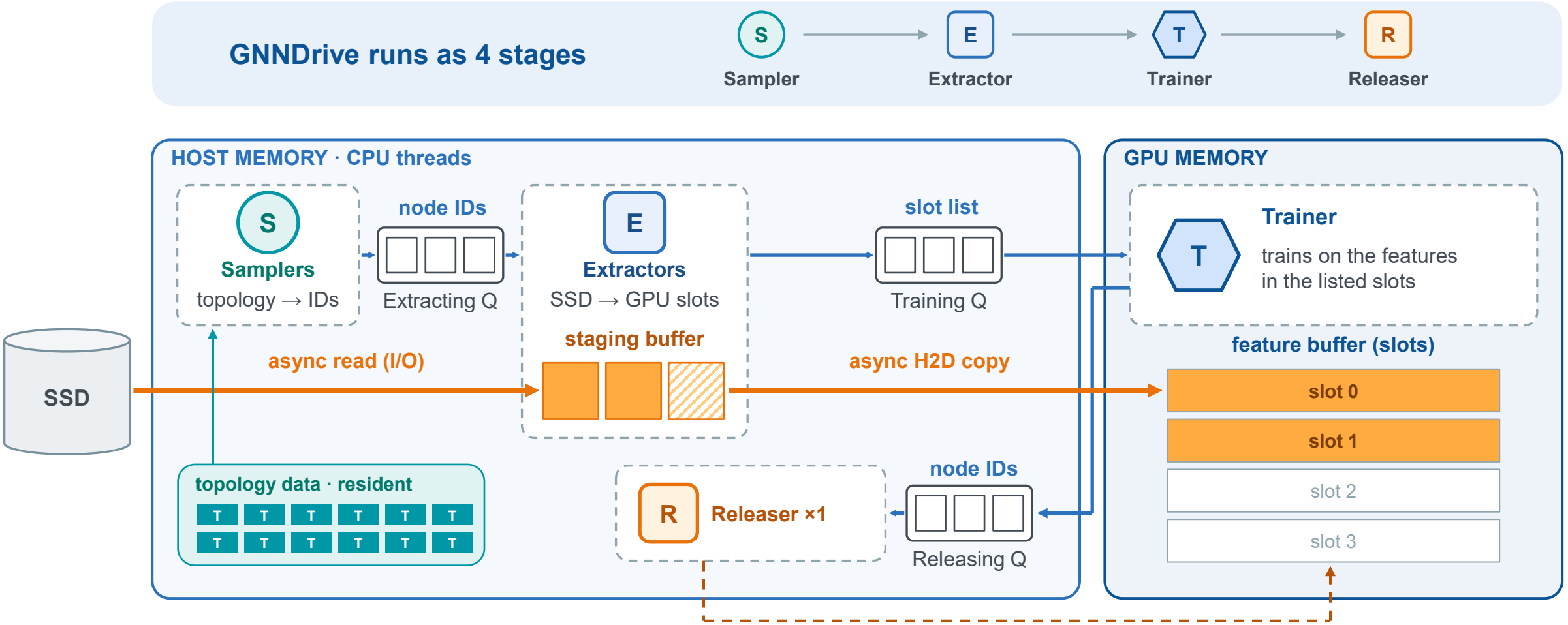
Add Queues so the Stages Run in Parallel

Queues between stages hold only node IDs, so every stage runs at its own pace.



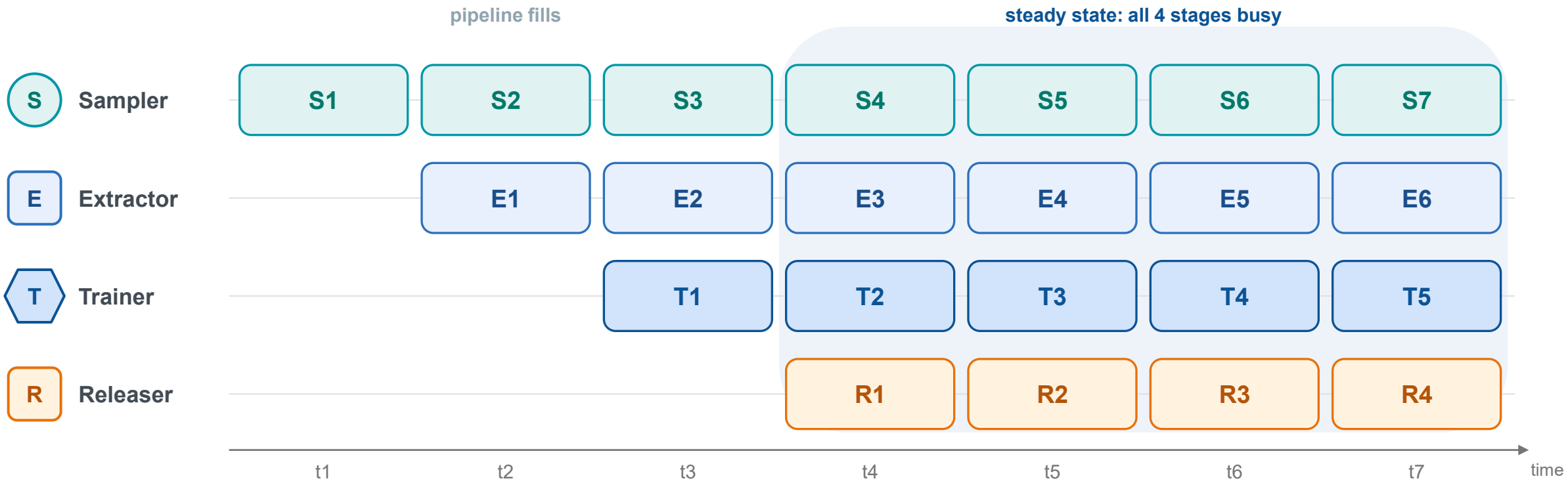
Four Stages Connected by Queues

Four independent stages linked by queues; features flow SSD → staging buffer → GPU feature buffer.



Pipelining: Four Stages Work on Different Mini-batches at Once

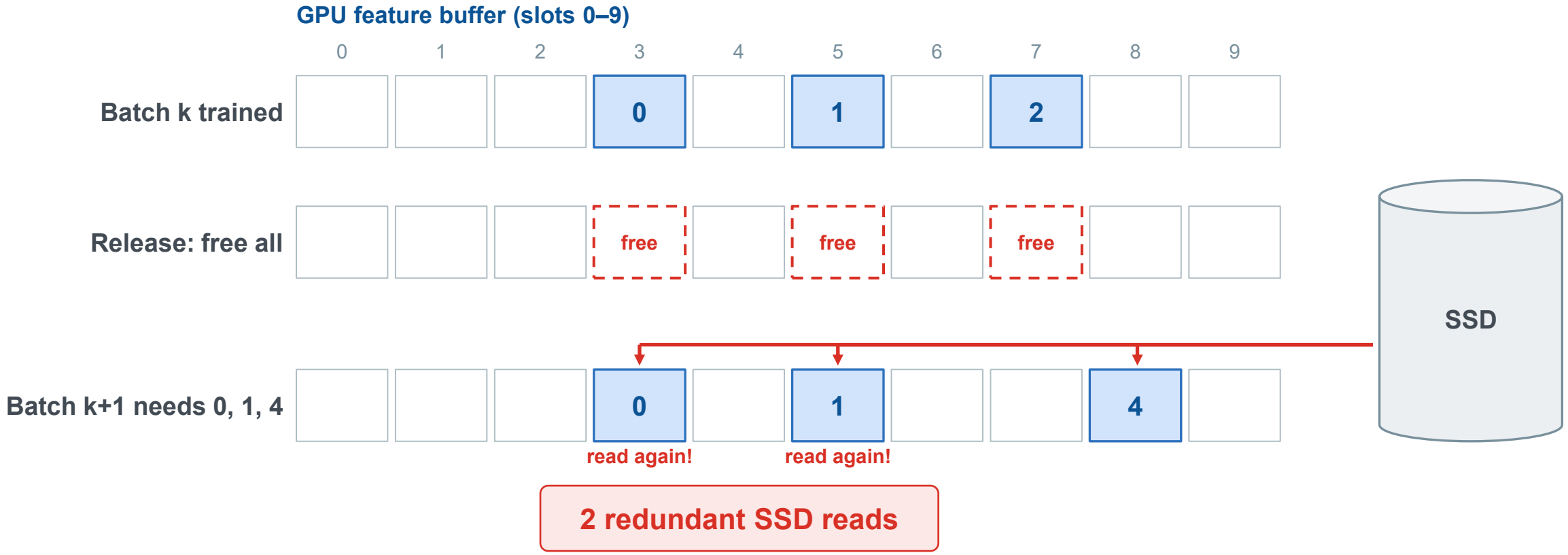
While the trainer runs batch 1, the extractor already loads batch 2 and the sampler starts batch 3.



From t4 on, a mini-batch finishes every step while SSD, CPU and GPU all stay busy.

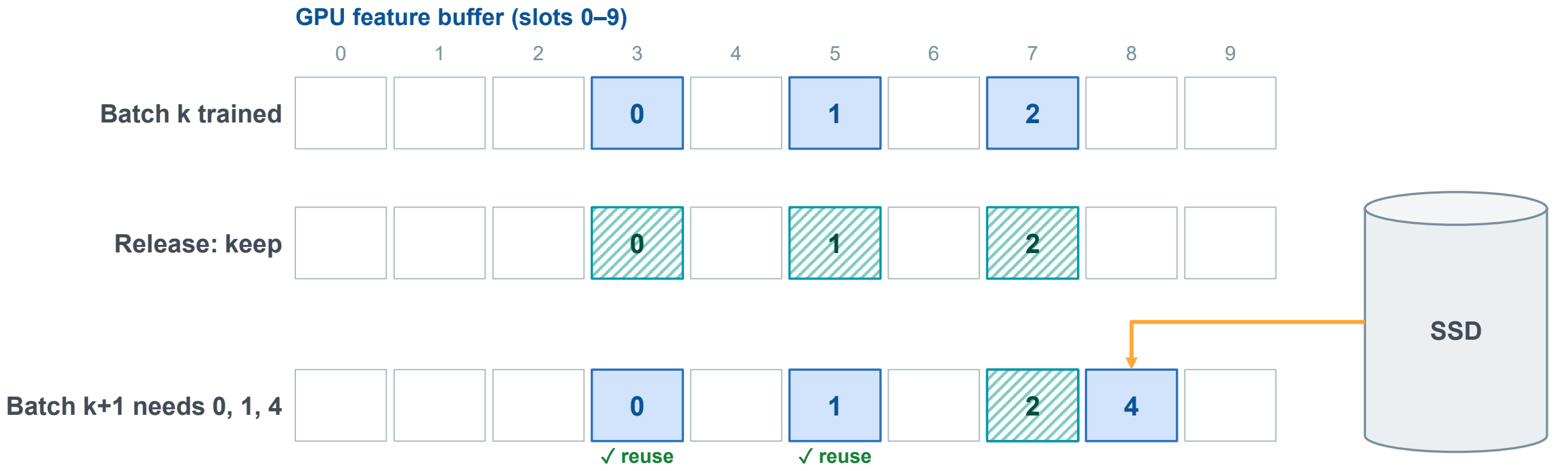
Freeing Everything Wastes I/O

Naively, each batch frees its slots. so the next batch re-reads overlapping nodes from SSD.



Freeing Everything Wastes I/O

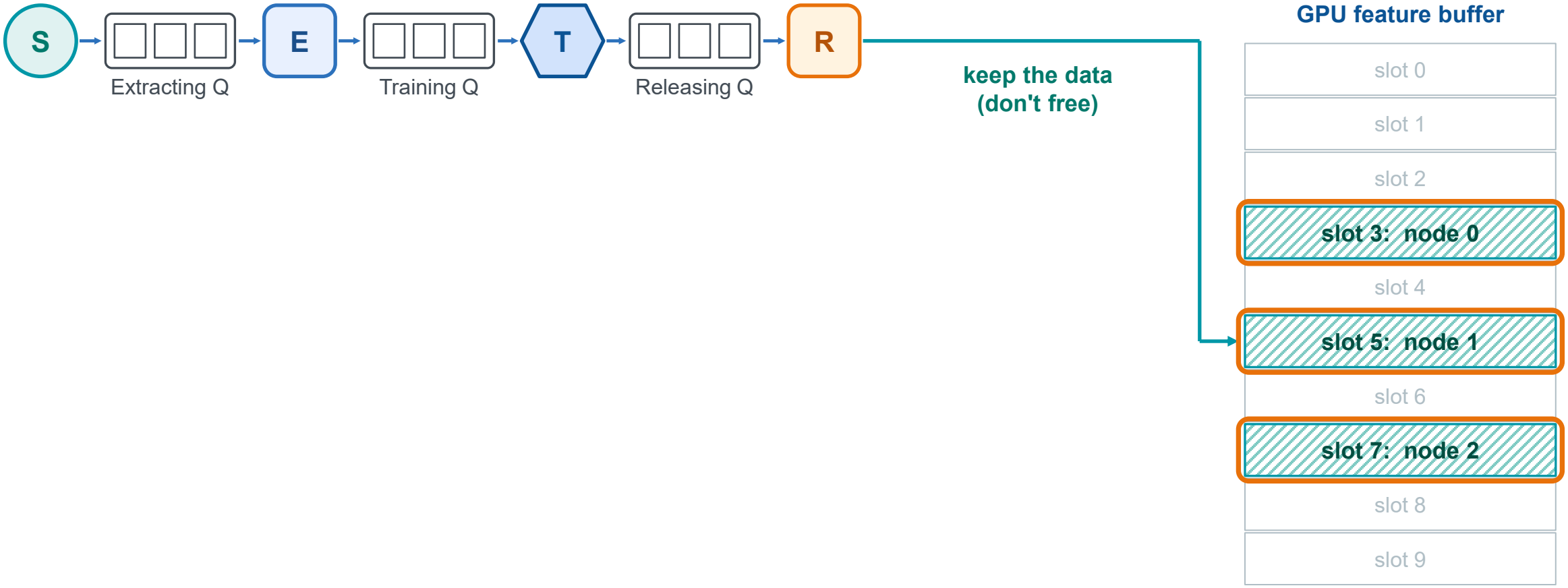
If finished features stayed in their slots, overlapping nodes could be reused.



1 SSD read

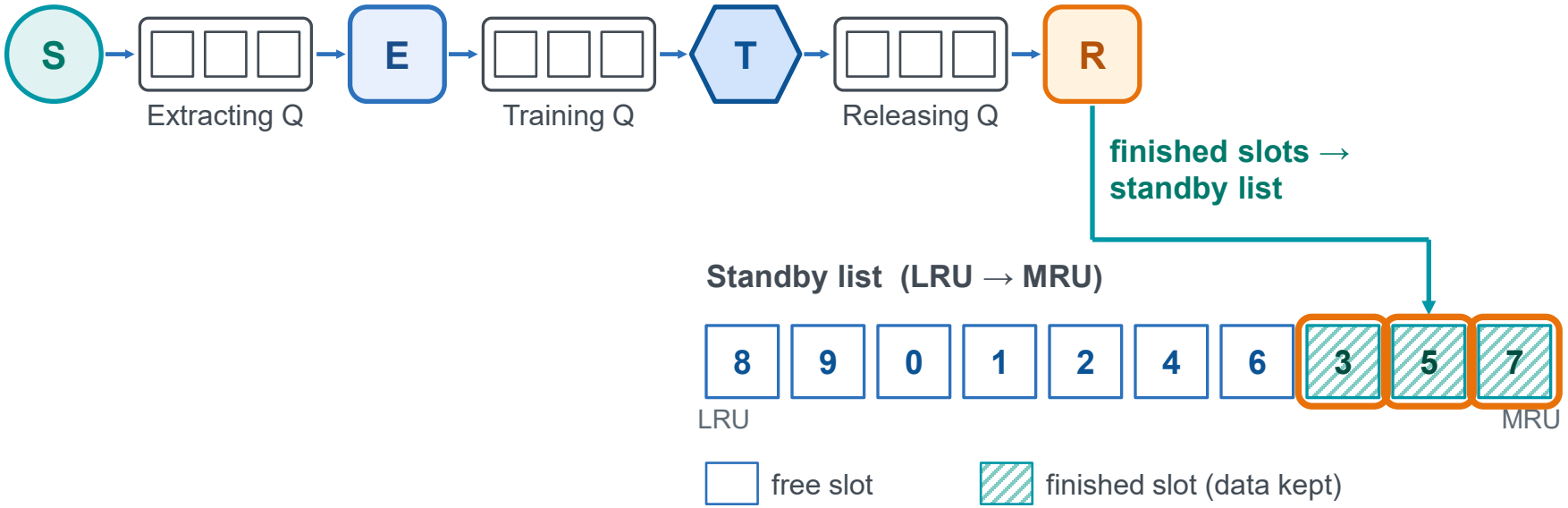
Keep Finished Features: Standby List + Mapping Table

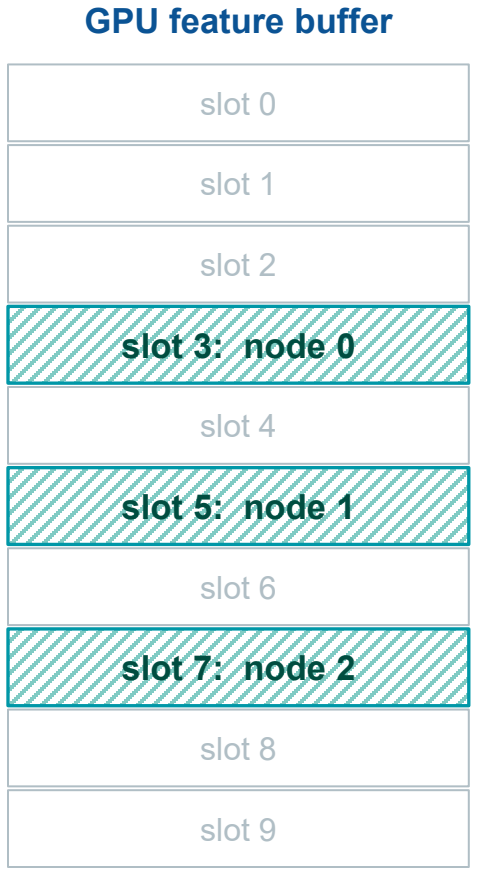
Release no longer frees data: finished features stay in their GPU slots.



Keep Finished Features: Standby List + Mapping Table

The buffer is finite → a standby list tracks slots that may be handed out : free + finished slots.





Keep Finished Features: Standby List + Mapping Table

The standby list knows only slots → a mapping table records which slot holds each node (−1 = none).



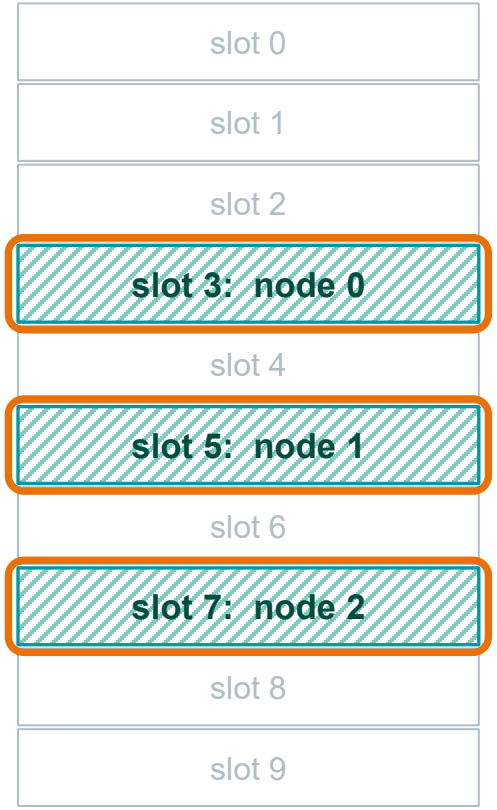
Mapping table

node	slot
node 0	**3**
node 1	**5**
node 2	**7**
node 3	-1
node 4	-1
node 5	-1

Standby list (LRU → MRU)



GPU feature buffer



Look Up Before Loading

A batch needs nodes 0, 1, 4 → 0 and 1 are in slots 3 and 5 (reuse); only node 4 (−1) is read.



batch needs 0 1 4 → **slots** 3 5 ?

Mapping table

node	slot
node 0	3
node 1	5
node 2	7
node 3	-1
node 4	-1
node 5	-1

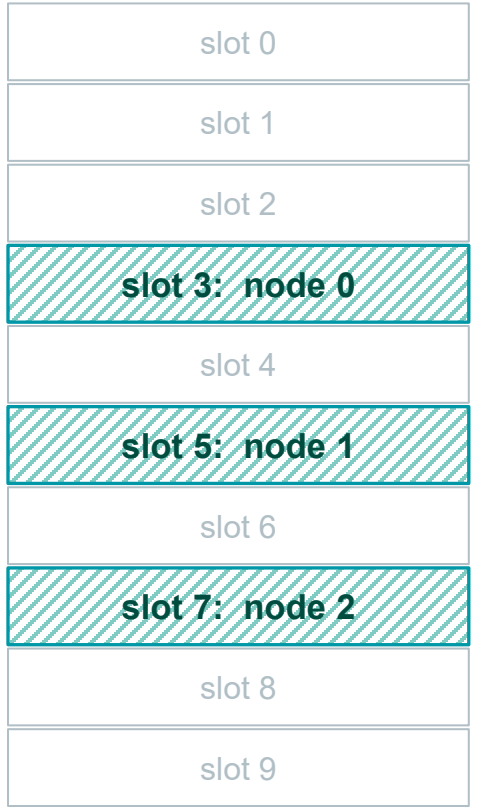
Standby list (LRU → MRU)



free slot finished slot (data kept)

0, 1: ✓ reuse · 4: −1 → I/O

GPU feature buffer



Look Up Before Loading

Node 4 takes the LRU slot (8) and the table says 4 → 8 at once — but its data arrives only later.



batch needs 0 1 4 → slots 3 5 8

Mapping table

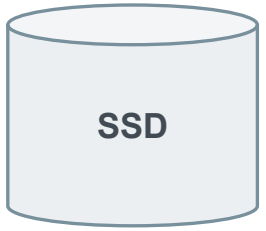
node	slot
node 0	3
node 1	5
node 2	7
node 3	-1
node 4	8
node 5	-1

1 First, assign a slot to node 4

Standby list (LRU → MRU)



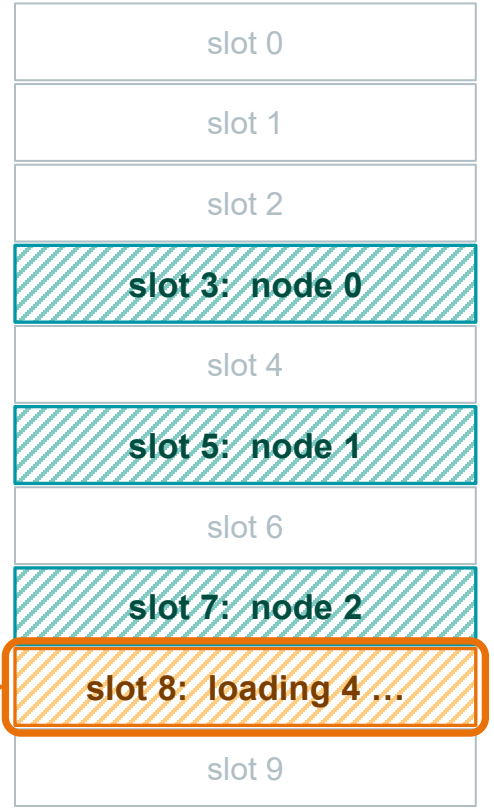
free slot finished slot (data kept)



2 Load node 4's features from SSD into the slot

async read + H2D (in flight)

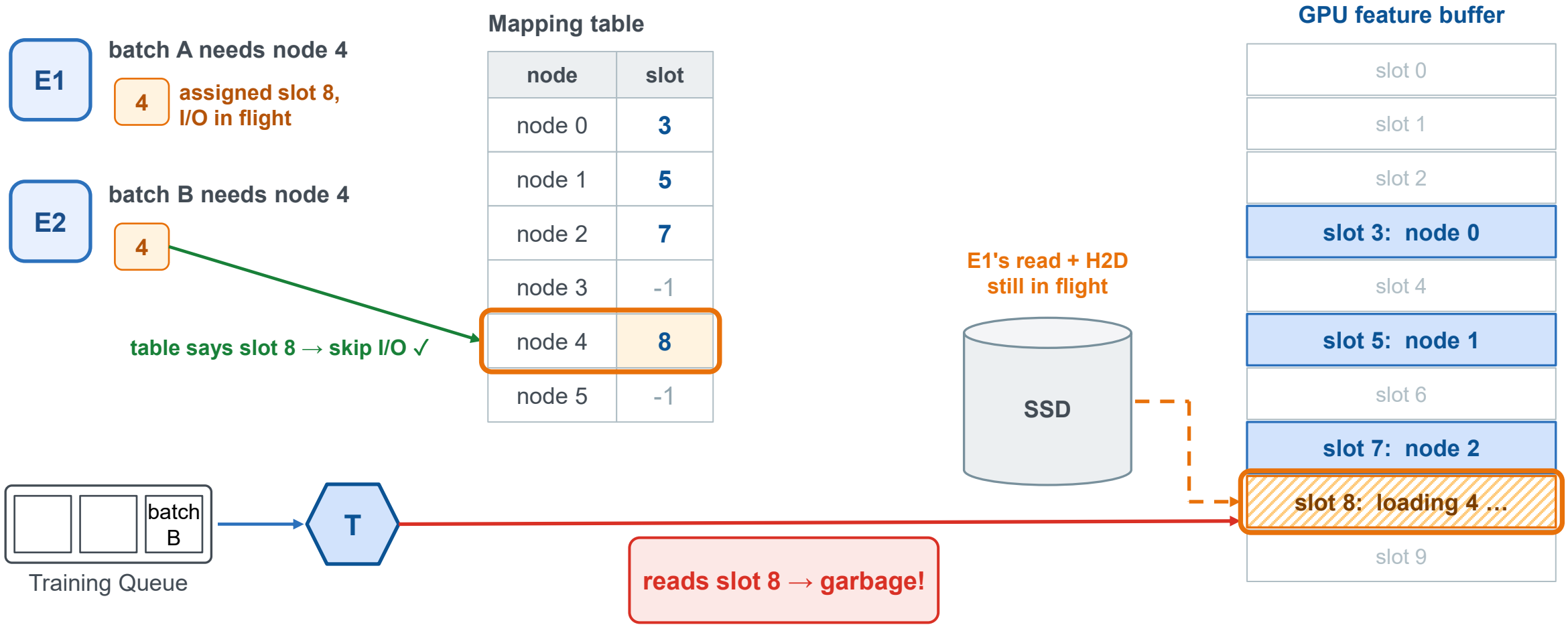
GPU feature buffer



slot 8: still empty

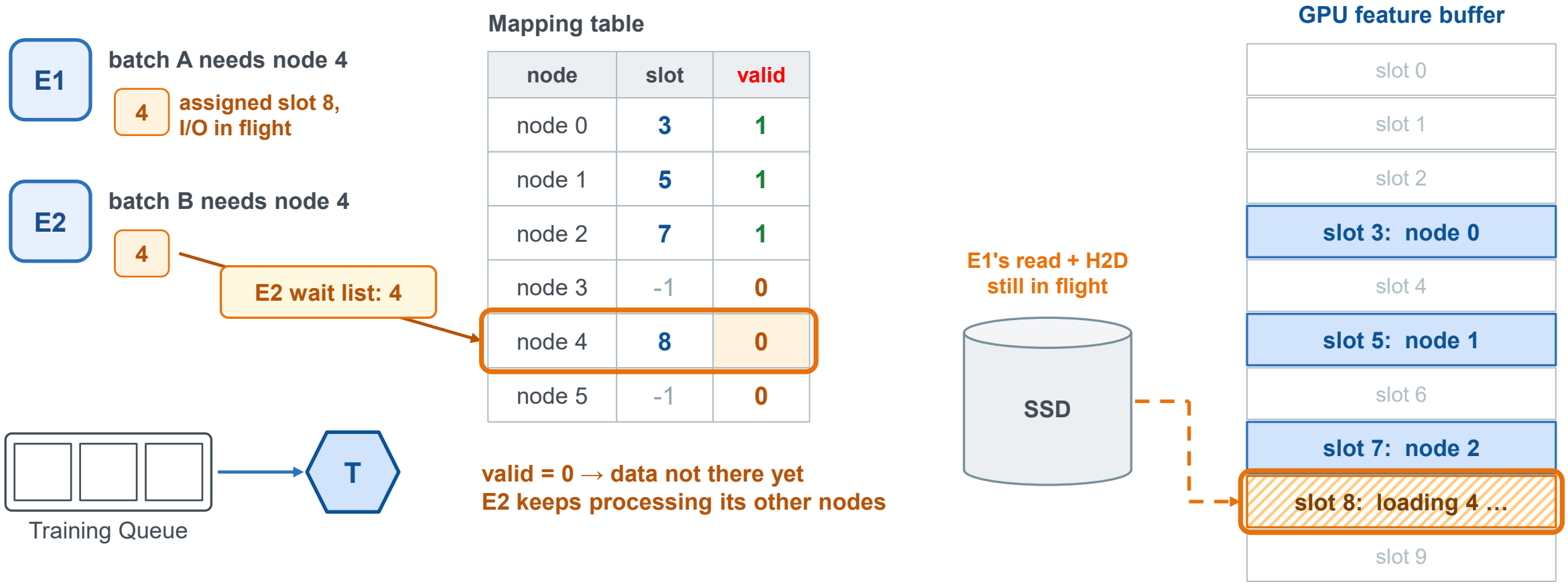
Problem: Out-of-Order Training Reads an Empty Slot

E2's batch is ready first and trains first (out of order) — but slot 8 is still empty.



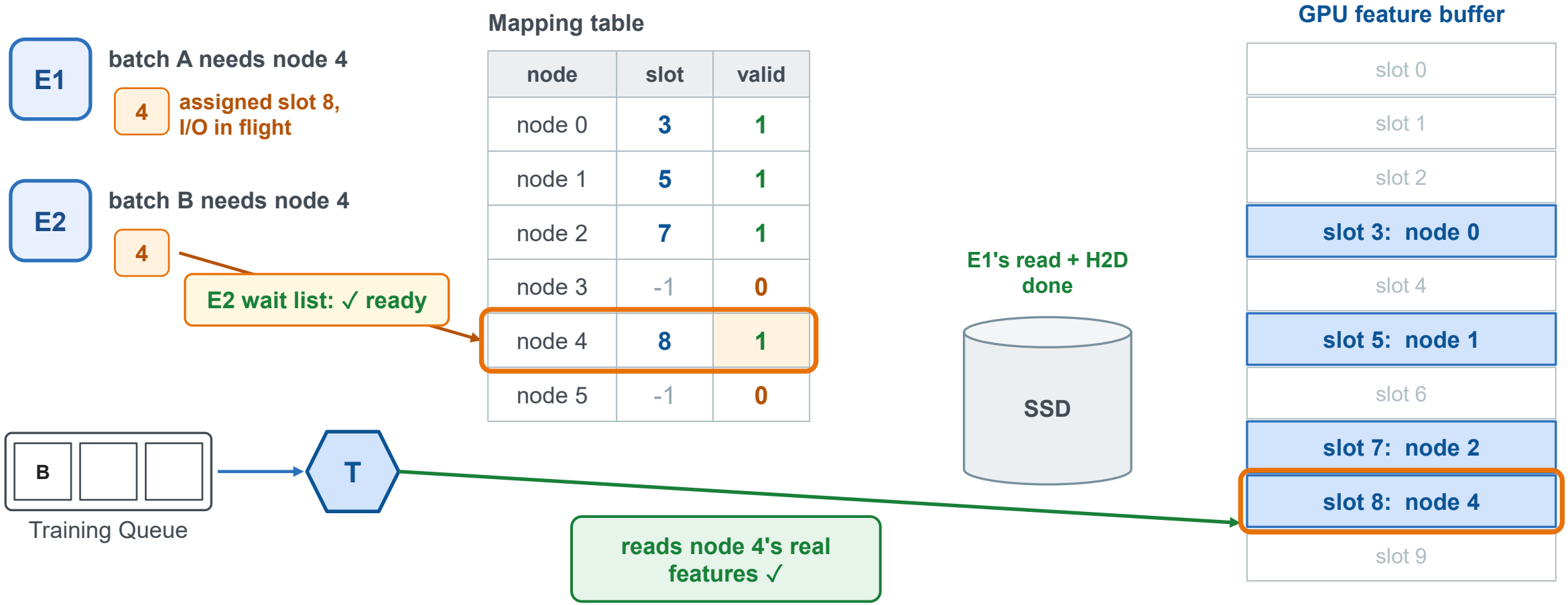
Fix: Valid Bit + Wait List

Add a valid bit (1 only after I/O + H2D finish). E2 sees valid = 0 → puts node 4 on its wait list.



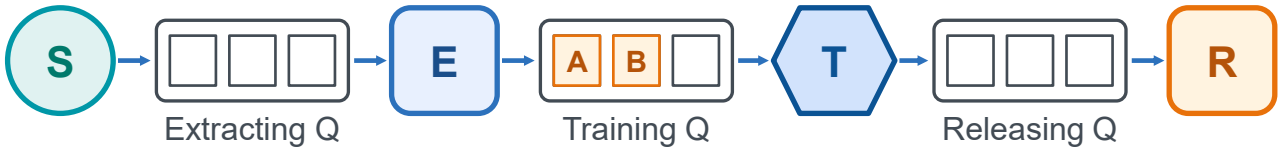
Fix: Valid Bit + Wait List

When E1's copy finishes, valid → 1. E2 re-checks its wait list at the end, then pushes its batch.



Architecture So Far, and a New Problem

Pipeline · GPU feature buffer · standby list · mapping table (slot, valid) · per-extractor wait lists.



batches A and B both use node 0 (slot 3)

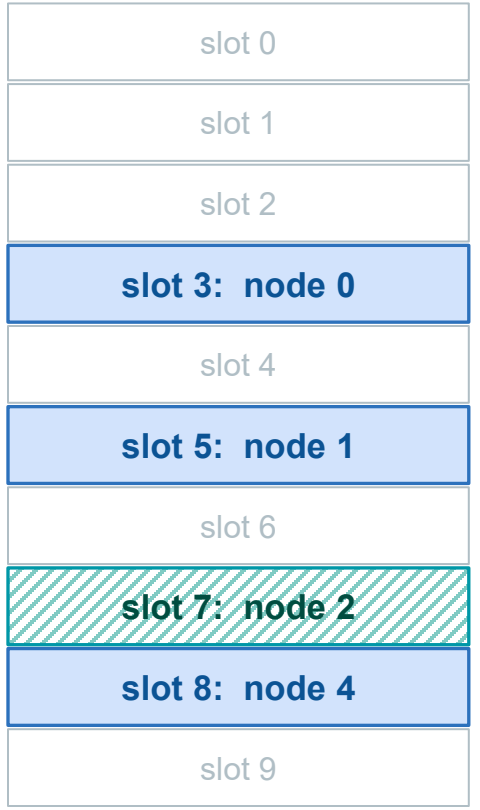
Mapping table

node	slot	valid
node 0	3	1
node 1	5	1
node 2	7	1
node 3	-1	0
node 4	8	1
node 5	-1	0

Standby list (LRU → MRU)

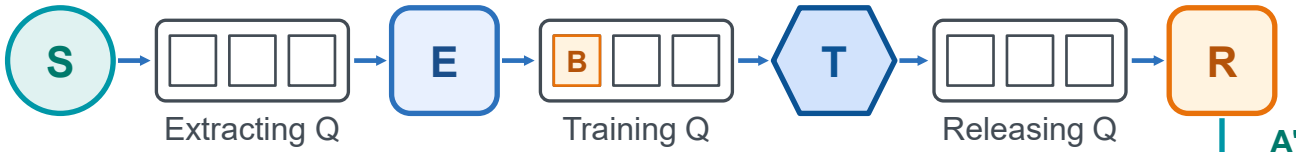


GPU feature buffer



Architecture So Far, and a New Problem

Batch A finishes → the releaser puts its slots (3, 5, 8) on the standby list — but B still needs slot 3!



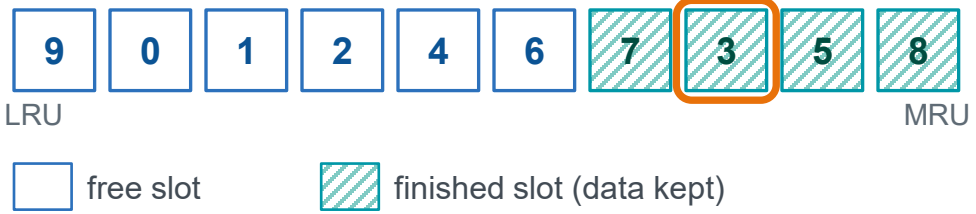
batch B (still queued) needs slot 3!

A's slots → standby list

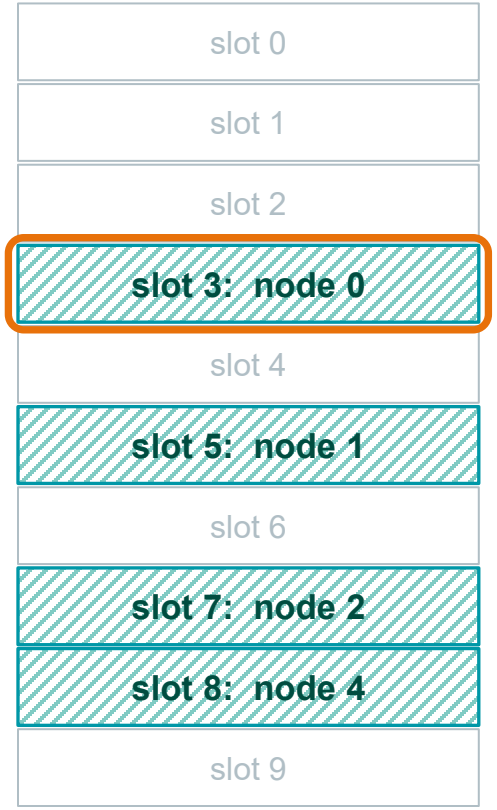
Mapping table

node	slot	valid
node 0	3	1
node 1	5	1
node 2	7	1
node 3	-1	0
node 4	8	1
node 5	-1	0

Standby list (LRU → MRU)



GPU feature buffer



Add a Reference Count: slot · ref · valid

ref = number of batches using the slot: +1 on extract, −1 on release. Hence replaceable only at ref = 0.



Mapping table

node	slot	ref	valid
node 0	3	1	1
node 1	5	0	1
node 2	7	0	1
node 3	-1	0	0
node 4	8	0	1
node 5	-1	0	0

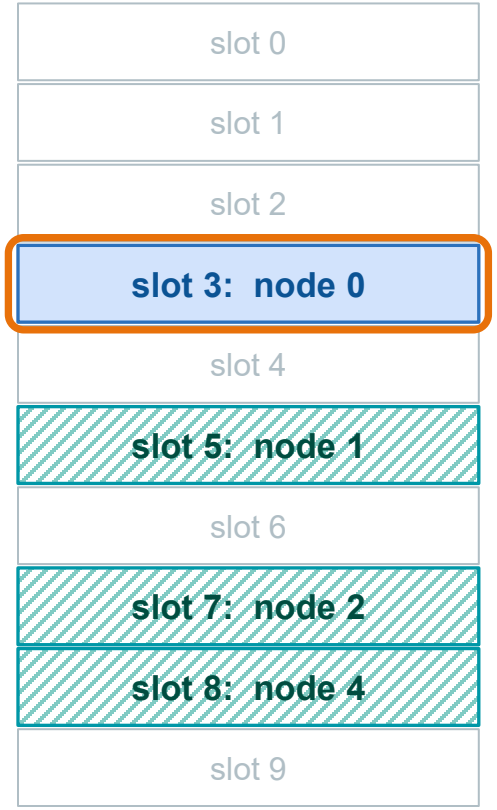
Standby list (LRU → MRU)



free slot finished slot (data kept)

slot 3 stays out of the standby list ✓
extract: ref + 1
release: ref − 1
only ref = 0 → standby list

GPU feature buffer



GNNDrive Full Process in Detail

A new batch needs nodes 7, 6, 1, 9, 5. This is the buffer's state right now.

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid	node	slot	ref	valid
node 0	3	0	1	node 5	-1	0	0
node 1	0	1	1	node 6	-1	0	0
node 2	-1	0	0	node 7	4	0	1
node 3	5	1	1	node 8	-1	0	0
node 4	-1	0	0	node 9	1	1	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: free
slot 3: node 0
slot 4: node 7
slot 5: node 3

Node alias list (slots)

Standby list



free slot

valid, unused slot

GNNDrive Full Process in Detail

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid	node	slot	ref	valid
node 0	3	0	1	node 5	-1	0	0
node 1	0	1	1	node 6	-1	0	0
node 2	-1	0	0	node 7	4	1	1
node 3	5	1	1	node 8	-1	0	0
node 4	-1	0	0	node 9	1	1	1

Feature buffer (GPU)

- slot 0: node 1
- slot 1: node 9
- slot 2: free
- slot 3: node 0
- slot 4: node 7
- slot 5: node 3

Node alias list (slots)

4

Standby list

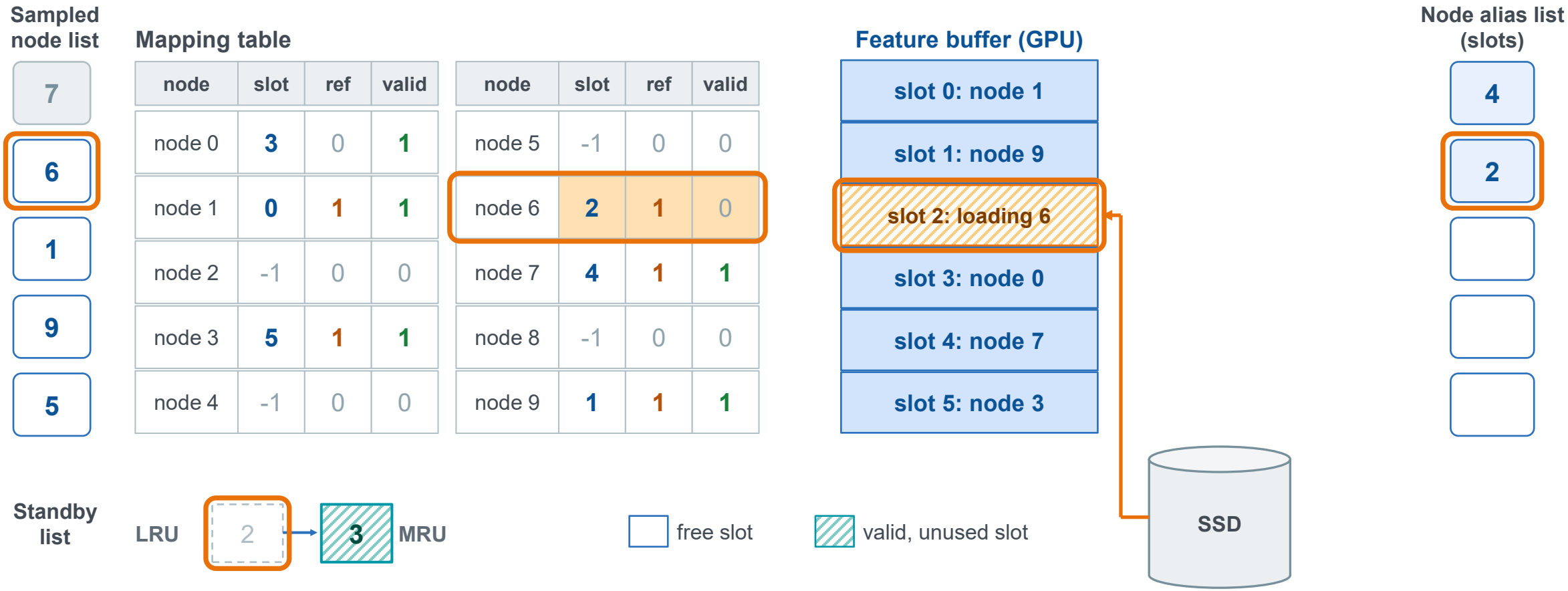
LRU — 2 → 3 → 4 — MRU

free slot

valid, unused slot

GNNDrive Full Process in Detail

Node 6: not in the buffer → take the LRU slot (2, free); ref 1, valid 0; load it asynchronously.



GNNDrive Full Process in Detail

Nodes 1 and 9: valid and already used by other batches (ref > 0) → just ref++ (1 → 2), no I/O.

Sampled node list

7

6

1

9

5

Mapping table

node	slot	ref	valid
node 0	3	0	1
node 1	0	2	1
node 2	-1	0	0
node 3	5	1	1
node 4	-1	0	0

node	slot	ref	valid
node 5	-1	0	0
node 6	2	1	0
node 7	4	1	1
node 8	-1	0	0
node 9	1	2	1

Feature buffer (GPU)

- slot 0: node 1
- slot 1: node 9
- slot 2: loading 6
- slot 3: node 0
- slot 4: node 7
- slot 5: node 3

Node alias list (slots)

4

2

0

1

Standby list

LRU [3] MRU

free slot

valid, unused slot

No Free Slot Left: Reverse Mapping Array

Node 5: not in the buffer — and no free slot remains. The LRU slot 3 still holds some node's data. Whose?

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid
node 0	3	0	1
node 1	0	2	1
node 2	-1	0	0
node 3	5	1	1
node 4	-1	0	0

node	slot	ref	valid
node 5	-1	0	0
node 6	2	1	0
node 7	4	1	1
node 8	-1	0	0
node 9	1	2	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: loading 6
slot 3: node 0
slot 4: node 7
slot 5: node 3

Node alias list (slots)

4
2
0
1

Standby list

LRU 3 MRU

free slot

valid, unused slot

No Free Slot Left: Reverse Mapping Array

A reverse mapping array (slot → node) answers instantly: slot 3 holds node 0.

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid
node 0	3	0	1
node 1	0	2	1
node 2	-1	0	0
node 3	5	1	1
node 4	-1	0	0

node	slot	ref	valid
node 5	-1	0	0
node 6	2	1	0
node 7	4	1	1
node 8	-1	0	0
node 9	1	2	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: loading 6
slot 3: node 0
slot 4: node 7
slot 5: node 3

Reverse map slot → node

1
9
6
0
7
3

Node alias list (slots)

4
2
0
1

Standby list LRU 3 MRU

free slot

valid, unused slot

Invalidate the Old Node, Assign the New One

Invalidate node 0's entry: slot → −1 and valid → 0 (its ref is already 0).

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid	node	slot	ref	valid
node 0	-1	0	0	node 5	-1	0	0
node 1	0	2	1	node 6	2	1	0
node 2	-1	0	0	node 7	4	1	1
node 3	5	1	1	node 8	-1	0	0
node 4	-1	0	0	node 9	1	2	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: loading 6
slot 3: node 0
slot 4: node 7
slot 5: node 3

Reverse map slot → node

1
9
6
0
7
3

Node alias list (slots)

4
2
0
1

Standby list

LRU 3 MRU

free slot

valid, unused slot

Invalidate the Old Node, Assign the New One

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid
node 0	-1	0	0
node 1	0	2	1
node 2	-1	0	0
node 3	5	1	1
node 4	-1	0	0

node	slot	ref	valid
node 5	3	1	0
node 6	2	1	0
node 7	4	1	1
node 8	-1	0	0
node 9	1	2	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: loading 6
slot 3: loading 5
slot 4: node 7
slot 5: node 3

Reverse map slot → node

1
9
6
5
7
3

Node alias list (slots)

4
2
0
1
3

SSD

Standby list

LRU 3 MRU

free slot

valid, unused slot

Hand Off to Training, Then Release

Loads finish → valid = 1 for nodes 6 and 5. The node alias list [4, 2, 0, 1, 3] goes to the Training Queue.

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid
node 0	-1	0	0
node 1	0	2	1
node 2	-1	0	0
node 3	5	1	1
node 4	-1	0	0

node	slot	ref	valid
node 5	3	1	1
node 6	2	1	1
node 7	4	1	1
node 8	-1	0	0
node 9	1	2	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: node 6
slot 3: node 5
slot 4: node 7
slot 5: node 3

Reverse map slot → node

1
9
6
5
7
3

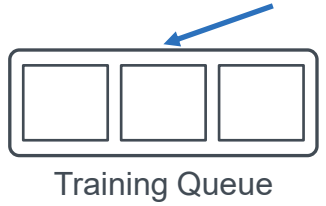
Node alias list (slots)

4
2
0
1
3

Standby list LRU *(empty)* MRU

free slot

valid, unused slot



Hand Off to Training, Then Release

Release: another batch finishes, node 3's ref drops 1 → 0 → slot 5 joins the MRU end of the standby list.

Sampled node list

7
6
1
9
5

Mapping table

node	slot	ref	valid	node	slot	ref	valid
node 0	-1	0	0	node 5	3	1	1
node 1	0	2	1	node 6	2	1	1
node 2	-1	0	0	node 7	4	1	1
node 3	5	0	1	node 8	-1	0	0
node 4	-1	0	0	node 9	1	2	1

Feature buffer (GPU)

slot 0: node 1
slot 1: node 9
slot 2: node 6
slot 3: node 5
slot 4: node 7
slot 5: node 3

Reverse map slot → node

1
9
6
5
7
3

Node alias list (slots)

4
2
0
1
3

Standby list LRU [5] MRU

free slot

valid, unused slot

GNNDrive Extract Stage at a Glance

Samplers → Extracting Q → Extractors (async load + transfer) → Training Q → Trainer → Releasing Q → Releaser.

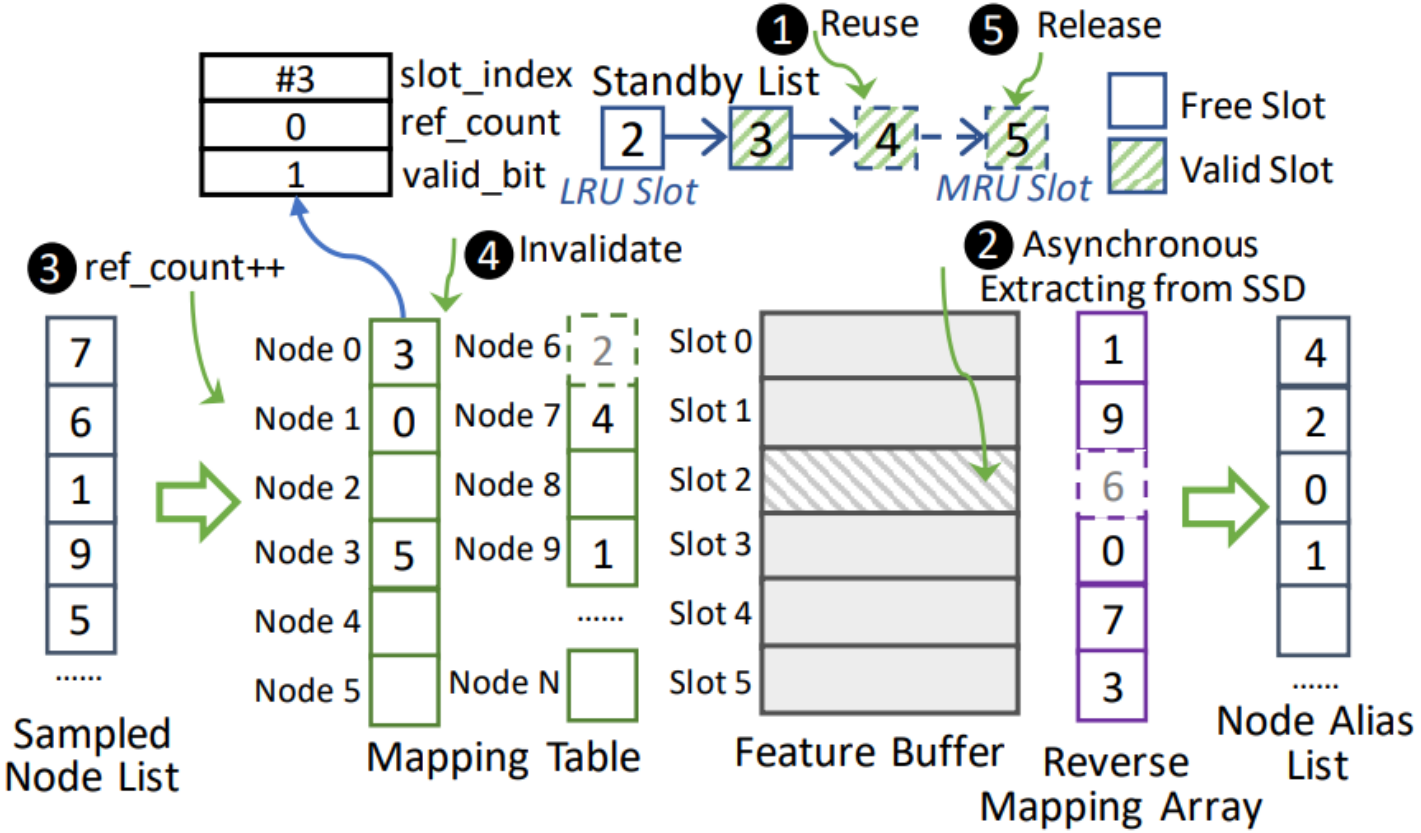


Figure 5: An example of GNNDrive's extract stage.

Evaluation Setup

One ordinary machine; host memory capped at 32 GB by default.

Hardware

- Intel Xeon Gold 6342
- NVIDIA RTX 3090 (24 GB)
- Samsung PM883 SSD
- **Host memory: 32 GB** (default)

Software

- Ubuntu 22.04 · Linux 5.19
- Python 3.8 · PyTorch 1.12
- CUDA 11.6 · PyG 2.2
- io_uring + CUDA async copies

Models & workload

- GraphSAGE · GCN · GAT — 3 layers, hidden 256
- fan-out (10, 10, 10); GAT (10, 10, 5)
- mini-batch 1,000
- **Metric:** average epoch time over 10 epochs

Baselines & GNNDrive

- PyG+ · Ginex (6 GB neighbor + 24 GB feature cache) · MariusGNN
- all baselines use 1 GPU
- **GNNDrive: 4 samplers · 4 extractors · 1 trainer · 1 releaser**
- variants: GPU-trained and CPU-trained

Evaluation Datasets

Dataset	#Nodes	#Edges	Dim.	#Classes	Topology	Features	Total
Papers100M	111 M	1.6 B	128	172	13 GB	53 GB	**67 GB**
Twitter	41.7 M	1.5 B	128	50	11 GB	20 GB	**31 GB**
Friendster	65.6 M	1.8 B	128	50	14 GB	32 GB	**46 GB**
MAG240M	122 M	1.3 B	768	153	10 GB	349 GB	**359 GB**

total > 32 GB host memory

Twitter: just under 32 GB

Sensitivity ① : Feature Dimension (64 → 512)

- All systems slow down as the dimension grows, **since more feature data must be loaded from SSD.**
- MAG240M + GraphSAGE, 64 → 512: **GPU-based GNNDrive 1.1× longer, PyG+ 7.0× longer**, because GNNDrive loads features asynchronously behind training, while PyG+'s larger features crowd the page cache sampling needs.
- With small dimensions (Twitter, Friendster), PyG+ and Ginex match or beat GNNDrive, **since the smaller feature files fit in the page cache / Ginex's feature cache.**

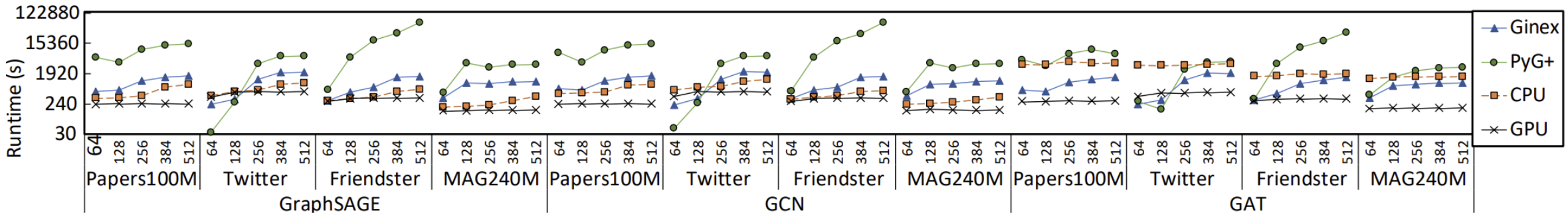


Figure 6: The runtime of one epoch in Ginex, PyG+, and GNNDrive with varying feature dimensions.

Sensitivity ② : Memory Capacity (8 → 128 GB)

- All systems run faster with more host memory, **which directly eases S–E memory contention**, since PyG+'s page cache and Ginex's caches can buffer more data. Ginex hits OOM at 8 GB (Twitter, GraphSAGE/GCN).
- Even at 8 GB, **GPU-based GNNDrive beats PyG+ by 5.8×**, because its extract stage takes only the memory it needs.
- Beyond 32 GB, GNNDrive barely speeds up, **since 32 GB already holds the entire topology** with its small staging buffer.

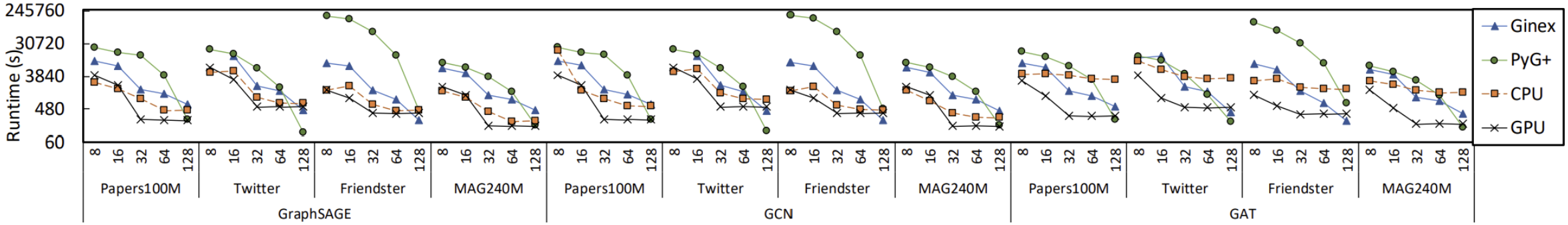


Figure 7: The runtime of one epoch in Ginex, PyG+, and GNNDrive with varying memory sizes.

Sensitivity ③ : Mini-batch Size (250 → 4,000)

- Larger batches generally cut epoch time for GNNDrive and Ginex, but **PyG+ fluctuates** (500 → 1,000: 43.2% longer on Twitter + GraphSAGE), because a larger batch needs more feature memory in its extract stage, which competes with sampling.
- **PyG+ runs out of memory at batch size 4,000** (Friendster, GAT).

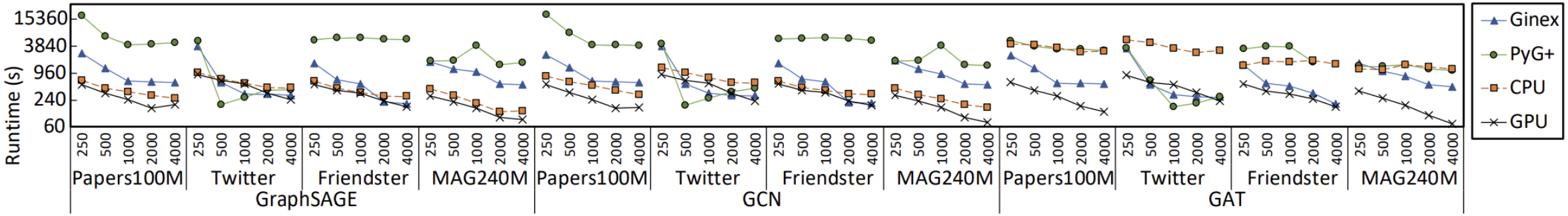


Figure 8: The runtime of one epoch in Ginex, PyG+, and GNNDrive with varying mini-batch sizes.

I/O Congestion Reduced

Unlike PyG+ and Ginex, **GNNDrive keeps I/O wait low with asynchronous I/Os**.

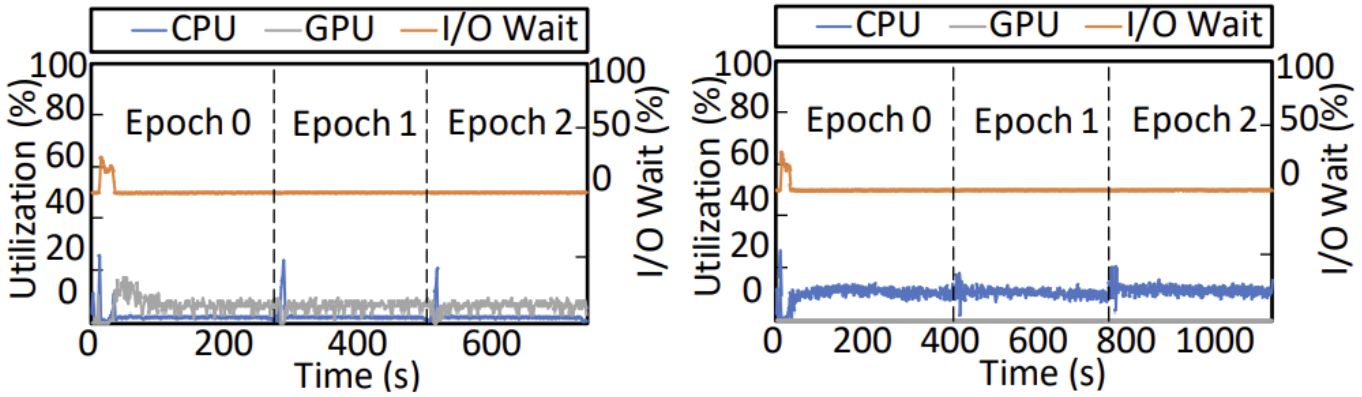
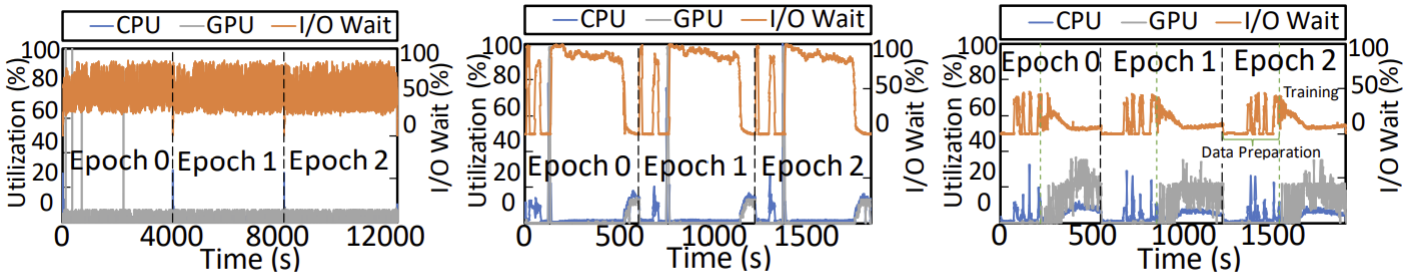
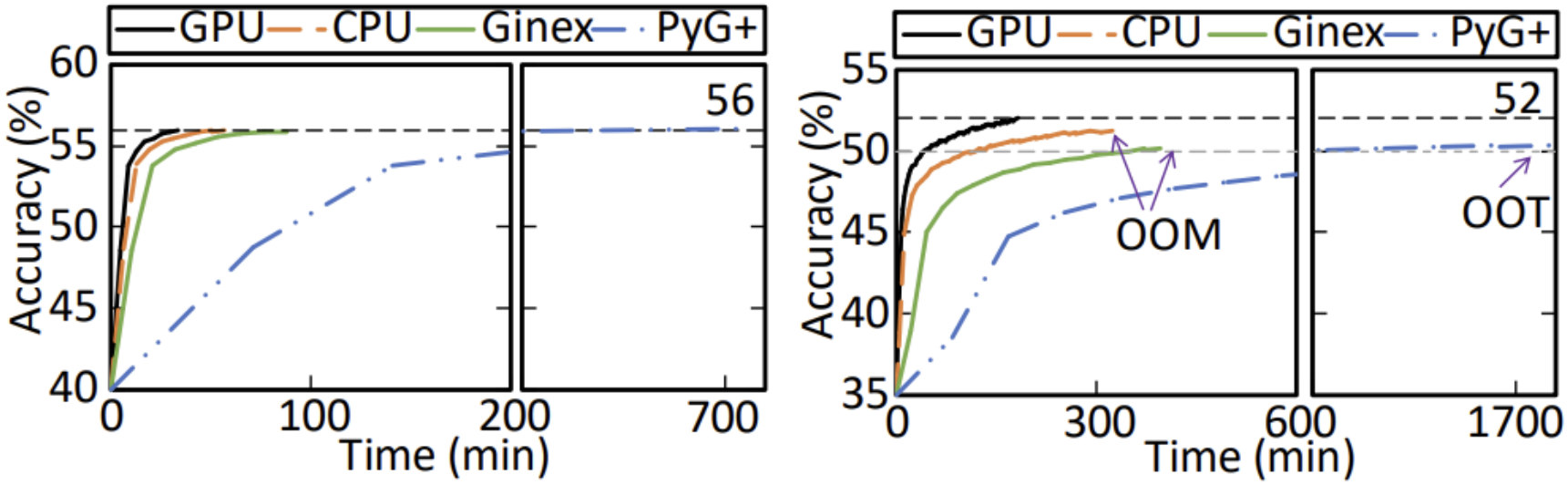


Figure 9: CPU utilization, GPU utilization and percentage of I/O wait time with GNNDrive for three epochs.



Convergence : Out-of-Order Batches Keep Accuracy

Reordering does not hurt convergence: **GNNDrive reaches the same accuracy far sooner.**



(a) Papars100M. (b) MAG240M.

Figure 12: A comparison of time-to-accuracy using Ginex, PyG+, and GNNDrive for training GraphSAGE model.

vs MariusGNN : No Costly Data Preparation

GNNDrive needs **no data preparation**; MariusGNN takes 2.7× longer and hits OOM.

Table 2: The runtime of one epoch in MariusGNN and GNNDrive (N/A: not applicable; OOM: out-of-memory.)

Runtime (s)	Data Preparation		Training		Overall	
Dataset	Papers-100M	MAG-240M	Papers-100M	MAG-240M	Papers-100M	MAG-240M
GNNDrive-GPU	N/A	N/A	241.12	166.66	241.12	166.66
GNNDrive-CPU	N/A	N/A	371.69	716.98	371.69	716.98
PyG+	N/A	N/A	4,115.73	5,037.61	4,115.73	5,037.61
Ginex	N/A	N/A	636.89	1,271.55	636.89	1,271.55
MariusGNN-32G	296.35	OOM	346.66	OOM	643.02	OOM
MariusGNN-128G	115.03	OOM	177.14	OOM	292.17	OOM

Conclusion

GNNDrive makes disk-based GNN training fast on one ordinary machine by fixing memory and I/O together.

Problem

- **Memory contention:** features evict the topology → PyG+ samples 5.4× slower
- **I/O congestion:** synchronous SSD reads stall CPU & GPU

Key ideas

- **Cap feature memory:** small fixed staging buffer + GPU feature buffer → topology stays resident
- **Async, decoupled pipeline:** 4 stages (SETR) + 3 queues, per-node async I/O, out-of-order batches

Results

- **16.9× / 2.6× / 2.7×** faster than PyG+ / Ginex / MariusGNN
- robust down to 8 GB of memory
- same accuracy, faster convergence · scales to multiple GPUs